

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

-----oOo-----



BÁO CÁO ĐỒ ÁN MÔN HỌC

MTH00051 - Toán Ứng Dụng và Thống Kê

ĐỒ ÁN 1: MA TRẬN VÀ TÍNH TOÁN KHOA HỌC

Giảng viên hướng dẫn : ThS. Võ Nam Thực Đoàn
ThS. Lê Nhựt Nam

Nhóm sinh viên thực hiện : Nhóm 1

STT	MSSV	Họ và tên
1	24120239	Phí Công Tuấn (Nhóm Trưởng)
2	24120260	Trương Tuấn Anh
3	24120295	Phạm Xuân Duy
4	24120301	Lâm Đông Hải
5	24120315	Phạm Huy Hoàng

TP. Hồ Chí Minh, tháng 04 năm 2026

MỤC LỤC

Danh mục hình ảnh	3
Danh mục bảng biểu	3
Danh mục mã giả	3
BẢNG PHÂN CÔNG CÔNG VIỆC	5
PHẦN 1: PHÉP KHỬ GAUSS VÀ CÁC ỨNG DỤNG	6
1.1. Cơ sở lý thuyết	6
1.1.1. Phương pháp khử Gauss và Gauss-Jordan	6
1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)	6
1.1.3. Tính định thức, ma trận nghịch đảo và hạng ma trận	7
1.2. Phân tích thuật toán	7
1.3. Cài đặt Python và Kiểm chứng	8
1.3.1. Cài đặt phép khử Gauss và Partial Pivoting	8
1.3.2. Tính định thức và Ma trận nghịch đảo	11
1.3.3. Xác định Hạng và Cơ sở	14
1.3.4. Kiểm thử với NumPy (Verification)	15
PHẦN 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA MANIM	18
2.1. Chéo hóa ma trận (Diagonalizable Matrix)	18
2.2. Kỹ thuật phân rã ma trận đã chọn	18
2.3. Cài đặt Python	19
2.3.1. Code thuật toán phân rã ma trận	19
2.3.2. Code thuật toán chéo hóa ma trận	25
2.4. Trực quan hóa toán học với Manim	30
2.4.1. Kịch bản video phân rã và chéo hóa	31
2.4.2. Ý nghĩa hình học và Ứng dụng thực tế	32
PHẦN 3: THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ	34
3.1. Giới thiệu tổng quan và Cơ sở lý thuyết	34
3.2. Cài đặt thuật toán giải hệ phương trình tuyến tính	35
3.2.1. Phương pháp Khử Gauss (Partial Pivoting)	35
3.2.2. Phương pháp Phân rã QR (Gram-Schmidt)	35



3.2.3. Phương pháp Lặp Gauss-Seidel	36
3.3. Thực nghiệm đo lường hiệu năng và phân tích	36
3.4. Phân tích tính ổn định số học: Ma trận Hilbert vs. SPD	39
KẾT LUẬN	42
1. Các kết quả đạt được	42
2. Khó khăn gặp phải	42
3. Bài học rút ra	43
TÀI LIỆU THAM KHẢO	44
PHỤ LỤC	45
Danh sách Link Video Demo	45

Danh mục hình ảnh

1	Mô phỏng mặt phẳng không gian con sinh bởi q_1, q_2 trong Manim	31
2	Hiệu ứng chuyển đổi vector riêng thành ma trận P và D	32
3	Đồ thị Log-Log mô phỏng trực quan thời gian chạy của ba phương pháp	37
4	Biểu đồ phân tích độ khuếch đại của số điều kiện Hilbert khi kích thước ma trận n gia tăng	40

Danh mục bảng biểu

1	Bảng đối chiếu sai số Định thức và Ma trận nghịch đảo	17
2	Thống kê sai số thuật toán Gram-Schmidt so với NumPy	24
3	Thống kê kết quả thuật toán Lập QR (Giới hạn: 1000 vòng lặp)	29
4	So sánh thời gian chạy thực tế (s) và sai số dư tương đối theo các hệ N	37
5	Phân tích biến động của sai số xoay quanh Số Điều kiện κ đối với hệ kích thước $N = 10$	39

Danh mục mã giả

1	Khử Gauss với Partial Pivoting (Chọn phần tử chốt một phần)	9
2	Quá trình Thế ngược (Back Substitution) - Phần 1: Tiền xử lý	10
3	Quá trình Thế ngược (Back Substitution) - Phần 2: Giải nghiệm	11
4	Tính định thức của ma trận qua phép khử Gauss	12
5	Tính ma trận nghịch đảo bằng phương pháp Gauss-Jordan	13
6	Xác định Hạng và Cơ sở của ma trận	14
7	Kiểm chứng tính đúng đắn và sai số thuật toán với NumPy	15
8	Phân Rã QR - Gram-Schmidt	20
9	Chéo Hóa Ma Trận - QR Iteration	26
10	Giải hệ phương trình bằng Khử Gauss (Partial Pivoting)	35
11	Giải hệ phương trình bằng Phân rã QR	35
12	Giải hệ phương trình bằng Lặp Gauss-Seidel	36

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	MSSV	Họ và tên	Nhiệm vụ	Đánh giá
1	24120239	Phí Công Tuấn	- Viết báo cáo LaTeX. - Code hàm gaussian_eliminate và back_substitution.	100%
2	24120260	Trương Tuấn Anh	- Trực quan hóa Manim (phân rã, chéo hóa). - Quay và render video demo_video.mp4.	100%
3	24120295	Phạm Xuân Duy	- Code Phần 2: Phân rã ma trận bằng thuật toán Gram-Schmidt và chéo hóa ma trận.	100%
4	24120301	Lâm Đông Hải	- Trực quan hóa Manim (phân rã, chéo hóa). - Quay và render video demo_video.mp4.	100%
5	24120315	Phạm Huy Hoàng	- Code hàm determinant, inverse, rank_and_basis và verify_solution.	100%

PHẦN 1: PHÉP KHỬ GAUSS VÀ CÁC ỨNG DỤNG

Nội dung phần này tập trung vào việc tự xây dựng (from scratch) phép khử Gauss có áp dụng kỹ thuật chọn phần tử chốt (Partial Pivoting) và các ứng dụng của nó trong giải tích ma trận.

1.1. Cơ sở lý thuyết

1.1.1. Phương pháp khử Gauss và Gauss-Jordan

Quá trình khử Gauss sử dụng ba phép biến đổi dòng sơ cấp:

- $R_i \leftrightarrow R_j$: Hoán đổi vị trí hai dòng i và j .
- $R_i \leftarrow c \cdot R_i$ ($c \neq 0$): Nhân một dòng với một hằng số khác không.
- $R_i \leftarrow R_i + c \cdot R_j$: Cộng một bội số của dòng này vào dòng khác.

Mục tiêu của phép khử Gauss là đưa ma trận về Dạng bậc thang dòng (Row Echelon Form - REF), trong đó các dòng toàn số 0 nằm dưới cùng và phần tử chỉ huy (pivot) của dòng dưới luôn nằm bên phải pivot của dòng trên. Phương pháp Gauss-Jordan tiến xa hơn bằng cách đưa ma trận về Dạng bậc thang dòng rút gọn (Reduced REF - RREF), yêu cầu thêm điều kiện mọi pivot phải bằng 1 và các phần tử khác trong cùng cột với pivot phải bằng 0.

1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Khi lập trình tính toán trên máy tính, thuật toán Gauss cơ bản dễ gặp lỗi chia cho 0 hoặc sai số làm tròn (round-off error) cực lớn nếu phần tử pivot quá nhỏ.

Để khắc phục, ta áp dụng kỹ thuật Partial Pivoting: Tại bước khử thứ k , thay vì lấy luôn a_{kk} làm pivot, ta tìm phần tử có giá trị tuyệt đối lớn nhất trong cột k từ dòng k đến dòng n :

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|$$

Sau đó, hoán đổi dòng k và dòng p trước khi thực hiện phép khử. Điều này đảm bảo các nhân tử luôn có trị tuyệt đối ≤ 1 , giúp duy trì tính ổn định số học.

1.1.3. Tính định thức, ma trận nghịch đảo và hạng ma trận

Tính định thức: Khi đưa ma trận vuông A về ma trận tam giác trên U bằng phép khử Gauss, định thức của A bằng tích các phần tử trên đường chéo chính của U , nhân với $(-1)^s$, trong đó s là số lần hoán đổi dòng đã thực hiện:

$$\det(A) = (-1)^s \cdot \prod_{i=1}^n u_{ii}$$

Tìm ma trận nghịch đảo: Một ma trận vuông A khả nghịch khi $\det(A) \neq 0$. Bằng phương pháp Gauss-Jordan, ta lập ma trận tăng cường ghép với ma trận đơn vị $[A|I_n]$. Khi thực hiện biến đổi sơ cấp đưa về trái về dạng RREF, vế phải sẽ trở thành ma trận nghịch đảo A^{-1} : $[A|I_n] \xrightarrow{\text{Gauss-Jordan}} [I_n|A^{-1}]$.

Hạng và Cơ sở không gian: Hạng (rank) của ma trận A chính là số dòng khác 0 trong dạng REF của nó (hoặc số pivot).

- **Cơ sở không gian cột ($\mathcal{C}(A)$):** Tập hợp các cột của ma trận A ban đầu tương ứng với các cột chứa pivot.
- **Cơ sở không gian dòng ($\mathcal{R}(A)$):** Tập hợp các dòng khác 0 trong ma trận REF.
- **Cơ sở không gian nghiệm ($\mathcal{N}(A)$):** Tập nghiệm cơ bản của hệ phương trình thuần nhất $Ax = 0$.

1.2. Phân tích thuật toán

Dưới đây là mô tả chi tiết các bước thực thi của Thuật toán khử Gauss có Partial Pivoting để giải hệ $Ax = b$:

Đầu vào: Ma trận $A \in \mathbb{R}^{n \times m}$, vector $b \in \mathbb{R}^n$.

Đầu ra: Nghiệm x , số lần hoán đổi dòng s .

Bước 1: Khởi tạo

Tạo ma trận tăng cường $M = [A|b]$ và đặt biến đếm số lần hoán đổi $s = 0$.

Bước 2: Quá trình khử tiến (Forward Elimination)

Lặp k từ 1 đến n :

- **Tìm Pivot:** Tìm chỉ số dòng p ($p \geq k$) sao cho $|M_{pk}|$ đạt giá trị lớn nhất. Nếu $|M_{pk}| < \epsilon$ (với ϵ rất nhỏ), cảnh báo ma trận suy biến.
- **Hoán đổi dòng:** Nếu $p \neq k$, thực hiện hoán đổi dòng $k \leftrightarrow p$ và tăng $s = s + 1$.

- *Khử các dòng dưới:* Lặp i từ $k + 1$ đến n . Tính nhân tử $l_{ik} = \frac{M_{ik}}{M_{kk}}$. Cập nhật dòng i : $M_i \leftarrow M_i - l_{ik} \cdot M_k$.

Bước 3: Quá trình thế ngược (Back Substitution)

Giải hệ tam giác trên từ dưới lên trên. Lặp i từ n lùi về 1:

$$x_i = \frac{1}{M_{ii}} \left(M_{i,m+1} - \sum_{j=i+1}^n M_{ij}x_j \right)$$

1.3. Cài đặt Python và Kiểm chứng

Trong phần này, nhóm sẽ trình bày mã nguồn cài đặt và các test case để chứng minh tính đúng đắn của thuật toán, có đối chiếu với thư viện NumPy.

1.3.1. Cài đặt phép khử Gauss và Partial Pivoting

Đoạn code sau cài đặt hàm `gaussian_eliminate(A, b)` - Trả về ma trận sau khi khử, nghiệm x , số lần hoán đổi:

Algorithm 1 Khử Gauss với Partial Pivoting (Chọn phần tử chốt một phần)

```
1: procedure GAUSSIAN_ELIMINATE( $A, b, verbose$ )
2:    $m \leftarrow$  Số hàng của  $A$ 
3:    $n \leftarrow$  Số cột của  $A$ 
4:   Khởi tạo số lần hoán đổi  $s \leftarrow 0$ , dung sai  $\epsilon \leftarrow 10^{-12}$ 
5:   Tạo ma trận tăng cường  $M \leftarrow [A|b]$  ▷ Kích thước  $m \times (n + 1)$ 
6:    $r \leftarrow 0$  ▷ Chỉ số hàng pivot hiện tại
7:   for  $j \leftarrow 0$  to  $n - 1$  do ▷ Duyệt qua từng cột
8:     if  $r \geq m$  then
9:       break ▷ Đã xét hết các hàng
10:    end if
11:    Tìm  $p \in [r, m - 1]$  sao cho  $|M[p][j]| = \max_{r \leq i < m} |M[i][j]|$ 
12:     $max\_val \leftarrow |M[p][j]|$ 
13:    if  $max\_val < \epsilon$  then
14:      continue ▷ Không có pivot, chuyển sang cột tiếp theo
15:    end if
16:    if  $p \neq r$  then
17:      Hoán đổi hàng  $r$  và hàng  $p$  của  $M$ :  $M[r] \leftrightarrow M[p]$ 
18:       $s \leftarrow s + 1$ 
19:    end if
20:    for  $i \leftarrow r + 1$  to  $m - 1$  do ▷ Khử các phần tử dưới pivot
21:       $factor \leftarrow M[i][j] / M[r][j]$ 
22:       $M[i][j] \leftarrow 0$  ▷ Gán bằng 0 để tránh sai số làm tròn
23:      for  $k \leftarrow j + 1$  to  $n$  do ▷ Khử đến cột cuối cùng (gồm cả  $b$ )
24:         $M[i][k] \leftarrow M[i][k] - factor \times M[r][k]$ 
25:      end for
26:    end for
27:     $r \leftarrow r + 1$  ▷ Chuyển sang hàng pivot tiếp theo
28:  end for
29:  Tách ma trận tam giác trên  $U$  và vector  $c$  từ ma trận tăng cường  $M$ 
30:   $x \leftarrow$  BACK_SUBSTITUTION( $U, c$ ) ▷ Gọi hàm thế ngược
31:  if  $verbose = \text{True}$  and  $x$  có vô số nghiệm then
32:    In công thức tổng quát của hệ phương trình
33:  end if
34:  return ( $M, x, s$ )
35: end procedure
```

Đoạn code sau cài đặt hàm `back_substitution(U, c)` - Giải hệ tam giác trên:

Algorithm 2 Quá trình Thế ngược (Back Substitution) - Phần 1: Tiền xử lý

```
1: procedure BACK_SUBSTITUTION( $U, c$ )
2:    $m, n \leftarrow$  Kích thước của ma trận tam giác trên  $U$ 
3:   Dung sai  $\epsilon \leftarrow 10^{-12}$ 
4:   Khởi tạo danh sách hàng chốt  $R_{piv} \leftarrow \emptyset$ , cột chốt  $C_{piv} \leftarrow \emptyset$ 
5:                                     ▷ Bước 1: Xác định vị trí các phần tử chốt
6:    $r \leftarrow 0$ 
7:   for  $j \leftarrow 0$  to  $n - 1$  do
8:     if  $r < m$  and  $|U[r][j]| > \epsilon$  then
9:       Thêm  $r$  vào  $R_{piv}$ , thêm  $j$  vào  $C_{piv}$ 
10:       $r \leftarrow r + 1$ 
11:     end if
12:   end for
13:                                     ▷ Bước 2: Kiểm tra tính có nghiệm của hệ
14:   for  $i \leftarrow r$  to  $m - 1$  do                                     ▷ Xét các dòng toàn số 0 của  $U$ 
15:     if  $|c[i]| > \epsilon$  then
16:       return "Hệ phương trình vô nghiệm"
17:     end if
18:   end for
19:                                     ▷ Bước 3: Xác định biến tự do và Khởi tạo ma trận nghiệm
20:   Tập các cột tự do  $F \leftarrow \{0, 1, \dots, n - 1\} \setminus C_{piv}$ 
21:    $k_{free} \leftarrow |F|$                                      ▷ Số lượng biến tự do
22:   Khởi tạo ma trận hệ số  $X_{coef}$  kích thước  $n \times (1 + k_{free})$  với các phần tử 0
23:   for mỗi chỉ số  $idx$  và cột  $f_j \in F$  do
24:      $X_{coef}[f_j][1 + idx] \leftarrow 1.0$                                      ▷ Gán hệ số 1 cho chính biến tự do đó
25:   end for
26:                                     ▷ – Thuật toán tiếp tục ở trang sau –
27: end procedure
```

Algorithm 3 Quá trình Thế ngược (Back Substitution) - Phần 2: Giải nghiệm

```
1: procedure BACK_SUBSTITUTION( $U, c$ ) ▷ Tiếp tục từ Phần 1
2: ▷ Bước 4: Thế ngược từ dưới lên trên
3:   for  $i \leftarrow |C_{piv}| - 1$  downto 0 do
4:      $pc \leftarrow C_{piv}[i], pr \leftarrow R_{piv}[i]$ 
5:     Khởi tạo mảng  $coef$  kích thước  $(1 + k_{free})$  với các phần tử 0
6:      $coef[0] \leftarrow c[pr]$  ▷ Gán hằng số ban đầu
7:     for  $j \leftarrow pc + 1$  to  $n - 1$  do
8:       if  $|U[pr][j]| > \epsilon$  then
9:         for  $k \leftarrow 0$  to  $k_{free}$  do
10:           $coef[k] \leftarrow coef[k] - U[pr][j] \times X_{coef}[j][k]$ 
11:        end for
12:      end if
13:    end for
14:     $piv \leftarrow U[pr][pc]$ 
15:     $X_{coef}[pc] \leftarrow coef / piv$  ▷ Lưu hệ số tham số của biến chốt
16:  end for
17: ▷ Bước 5: Trả về kết quả
18:  if  $k_{free} = 0$  then
19:    return  $X_{coef}[:, 0]$  ▷ Nghiệm duy nhất (chỉ lấy cột hằng số)
20:  else
21:    return  $(X_{coef}, F)$  ▷ Vô số nghiệm (trả về ma trận tham số)
22:  end if
23: end procedure
```

1.3.2. Tính định thức và Ma trận nghịch đảo

Đoạn code sau cài đặt hàm `determinant(A)` - Tính $\det(A)$ qua khử Gauss.

Algorithm 4 Tính định thức của ma trận qua phép khử Gauss

```
1: procedure DETERMINANT( $A$ )
2:    $n \leftarrow$  Số lượng hàng của ma trận  $A$ 
3:   if Có bất kỳ hàng nào của  $A$  có độ dài khác  $n$  then
4:     raise Lỗi "Định thức chỉ xác định cho ma trận vuông."
5:   end if
6:   Khởi tạo mảng  $b\_dummy \leftarrow [0.0, 0.0, \dots, 0.0]$  gồm  $n$  phần tử
7:   try:
8:      $(M_{aug}, s) \leftarrow$  GAUSSIAN_ELIMINATE( $A, b\_dummy, \text{False}$ )
9:      $det \leftarrow (-1)^s$  ▷ Dấu phụ thuộc vào số lần hoán đổi dòng  $s$ 
10:    for  $i \leftarrow 0$  to  $n - 1$  do
11:       $det \leftarrow det \times M_{aug}[i][i]$  ▷ Tích các phần tử trên đường chéo
12:    end for
13:    if  $|det| < 10^{-12}$  then
14:      return 0.0 ▷ Tránh sai số cực nhỏ hoặc  $-0.0$ 
15:    else
16:      return  $det$ 
17:    end if
18:  except: ▷ Bắt lỗi nếu ma trận suy biến không thể khử
19:    return 0.0
20: end procedure
```

Đoạn code sau cài đặt hàm `inverse(A)` - Tính A^{-1} bằng phương pháp Gauss–Jordan.

Algorithm 5 Tính ma trận nghịch đảo bằng phương pháp Gauss-Jordan

```

1: procedure INVERSE( $A$ )
2:    $n \leftarrow$  Số lượng hàng của ma trận  $A$ 
3:   if Có bất kỳ hàng nào của  $A$  có độ dài khác  $n$  then
4:     raise Lỗi "Chỉ ma trận vuông mới có khả năng nghịch đảo."
5:   end if
6:                                                                  $\triangleright$  Bước 1: Tạo ma trận ghép  $[A|I]$ 
7:   Khởi tạo ma trận đơn vị  $I$  kích thước  $n \times n$ 
8:   Tạo ma trận tăng cường  $M \leftarrow [A|I]$                                                                   $\triangleright$  Kích thước  $n \times 2n$ 
9:                                                                  $\triangleright$  Bước 2: Phép khử Gauss-Jordan
10:  for  $k \leftarrow 0$  to  $n - 1$  do
11:                                                                  $\triangleright$  a. Chọn phần tử chốt (Partial Pivoting)
12:      $max\_idx \leftarrow k$ 
13:     for  $i \leftarrow k + 1$  to  $n - 1$  do
14:       if  $|M[i][k]| > |M[max\_idx][k]|$  then
15:          $max\_idx \leftarrow i$ 
16:       end if
17:     end for
18:                                                                  $\triangleright$  b. Kiểm tra điều kiện tồn tại pivot
19:     if  $|M[max\_idx][k]| < 10^{-12}$  then
20:       return "Thông báo: Không có pivot tại cột  $k + 1$ . Ma trận không khả nghịch."
21:     end if
22:     Hoán đổi hàng  $k$  và hàng  $max\_idx$  của  $M$ :  $M[k] \leftrightarrow M[max\_idx]$ 
23:                                                                  $\triangleright$  c. Chuẩn hóa dòng  $k$ : đưa phần tử chốt về 1
24:      $piv \leftarrow M[k][k]$ 
25:     for  $j \leftarrow k$  to  $2n - 1$  do
26:        $M[k][j] \leftarrow M[k][j] / piv$ 
27:     end for
28:                                                                  $\triangleright$  d. Khử tất cả phần tử khác trên cột  $k$  về 0 (cả trên và dưới)
29:     for  $i \leftarrow 0$  to  $n - 1$  do
30:       if  $i \neq k$  then
31:          $factor \leftarrow M[i][k]$ 
32:          $M[i][k] \leftarrow 0$                                                                   $\triangleright$  Tránh sai số làm tròn nhỏ
33:         for  $j \leftarrow k + 1$  to  $2n - 1$  do
34:            $M[i][j] \leftarrow M[i][j] - factor \times M[k][j]$ 
35:         end for
36:       end if
37:     end for
38:   end for
39:                                                                  $\triangleright$  Bước 3: Tách ma trận nghịch đảo từ nửa phải
40:   Trích xuất các cột từ  $n$  đến  $2n - 1$  của  $M$  vào ma trận  $A^{-1}$ 
41:   return  $A^{-1}$ 
42: end procedure

```

1.3.3. Xác định Hạng và Cơ sở

Algorithm 6 Xác định Hạng và Cơ sở của ma trận

```

1: procedure RANK_AND_BASIS( $A$ )
2:    $m, n \leftarrow$  Kích thước của ma trận  $A$ 
3:   Dung sai  $\epsilon \leftarrow 10^{-12}$ 
4:    $\triangleright$  Bước 1: Đưa ma trận  $A$  về dạng Bậc thang (REF)
5:   Tạo vector  $b\_zero \leftarrow [0.0, 0.0, \dots, 0.0]$  gồm  $m$  phần tử
6:    $(M_{aug}, \_) \leftarrow \text{GAUSSIAN\_ELIMINATE}(A, b\_zero, \text{False})$ 
7:   Trích xuất ma trận  $U$  bằng cách bỏ đi cột cuối cùng ( $b$ ) của  $M_{aug}$ 
8:    $\triangleright$  Bước 2: Xác định cột pivot và hạng
9:   Khởi tạo danh sách các cột chốt  $pivot\_cols \leftarrow \emptyset$ 
10:   $r \leftarrow 0$ 
11:  for  $j \leftarrow 0$  to  $n - 1$  do
12:    if  $r < m$  and  $|U[r][j]| > \epsilon$  then
13:      Thêm  $j$  vào  $pivot\_cols$ 
14:       $r \leftarrow r + 1$ 
15:    end if
16:  end for
17:   $rank \leftarrow |pivot\_cols|$   $\triangleright$  Số lượng cột chốt chính là hạng của ma trận
18:   $\triangleright$  Bước 3: Xác định cơ sở không gian dòng  $\mathcal{R}(A)$ 
19:   $row\_basis \leftarrow rank$  dòng đầu tiên của ma trận  $U$ 
20:   $\triangleright$  Bước 4: Xác định cơ sở không gian cột  $\mathcal{C}(A)$ 
21:   $col\_basis \leftarrow$  Lấy các cột từ ma trận GỐC  $A$  dựa trên các chỉ số trong  $pivot\_cols$ 
22:   $\triangleright$  Bước 5: Xác định cơ sở không gian nghiệm  $\mathcal{N}(A)$ 
23:  Tập các cột tự do  $free\_cols \leftarrow \{0, 1, \dots, n - 1\} \setminus pivot\_cols$ 
24:  Khởi tạo danh sách cơ sở nghiệm  $null\_basis \leftarrow \emptyset$ 
25:  for mỗi chỉ số  $f_j \in free\_cols$  do
26:    Khởi tạo vector  $x \leftarrow [0.0, \dots, 0.0]$  gồm  $n$  phần tử
27:     $x[f_j] \leftarrow 1.0$   $\triangleright$  Gán giá trị 1 cho biến tự do đang xét
28:    for  $i \leftarrow rank - 1$  downto 0 do
29:       $pc \leftarrow pivot\_cols[i]$ 
30:       $val \leftarrow \sum_{k=pc+1}^{n-1} (U[i][k] \times x[k])$ 
31:       $x[pc] \leftarrow -val / U[i][pc]$   $\triangleright$  Thế ngược với vế phải  $b = 0$ 
32:    end for
33:    Thêm vector  $x$  vào  $null\_basis$ 
34:  end for
35:  return ( $rank, col\_basis, row\_basis, null\_basis$ )
36: end procedure

```

1.3.4. Kiểm thử với NumPy (Verification)

Algorithm 7 Kiểm chứng tính đúng đắn và sai số thuật toán với NumPy

```
1: procedure VERIFY_SOLUTION( $A, x, b$ )
2:   In ra màn hình dải phân cách "KIỂM CHỨNG VỚI NUMPY"
3:    $\triangleright$  1. Kiểm tra sai số nghiệm của hệ phương trình  $Ax = b$ 
4:   if  $x$  là một vector số thực (nghiệm duy nhất) then
5:      $residual \leftarrow ||A \times x - b||_2$   $\triangleright$  Tính chuẩn khoảng cách (Norm)
6:     In ra "Sai số hệ phương trình:  $residual$ "
7:   else if  $x$  là một tuple chứa ma trận tham số (vô số nghiệm) then
8:      $x_{coef}, free\_cols \leftarrow x$ 
9:     Trích xuất nghiệm đặc biệt  $x_0$  bằng cách cho tất cả các tham số tự do  $t_i = 0$ 
10:     $residual \leftarrow ||A \times x_0 - b||_2$ 
11:    In ra "Sai số nghiệm đặc biệt ( $t_i = 0$ ):  $residual$ "
12:   else
13:     In ra thông báo  $x$  (Ví dụ: "Hệ phương trình vô nghiệm")
14:   end if
15:    $\triangleright$  2. Kiểm tra định thức
16:   if Số hàng của  $A$  bằng Số cột của  $A$  then
17:      $det\_self \leftarrow \text{DETERMINANT}(A)$ 
18:      $det\_np \leftarrow \text{NumPy.det}(A)$ 
19:     In ra so sánh giữa  $det\_self$  và  $det\_np$ 
20:   end if
21:    $\triangleright$  3. Kiểm tra hạng ma trận
22:    $(rank\_self, \_, \_) \leftarrow \text{RANK\_AND\_BASIS}(A)$ 
23:    $rank\_np \leftarrow \text{NumPy.matrix\_rank}(A)$ 
24:   In ra so sánh giữa  $rank\_self$  và  $rank\_np$ 
25:    $\triangleright$  4. Kiểm tra ma trận nghịch đảo
26:   if Số hàng của  $A$  bằng Số cột của  $A$  and  $|det\_np| > 10^{-12}$  then
27:      $inv\_self\_res \leftarrow \text{INVERSE}(A)$ 
28:     if  $inv\_self\_res$  là một ma trận then
29:       Lấy ma trận nghịch đảo chuẩn  $inv\_np \leftarrow \text{NumPy.inv}(A)$ 
30:        $inv\_error \leftarrow ||inv\_self\_res - inv\_np||_F$   $\triangleright$  Tính chuẩn Frobenius
31:       In ra "Sai số ma trận nghịch đảo:  $inv\_error$ "
32:     else
33:       In ra thông báo  $inv\_self\_res$ 
34:     end if
35:   end if
36:   In ra màn hình dải phân cách kết thúc
37: end procedure
```

Để chứng minh tính đúng đắn và độ ổn định của các hàm tự cài đặt (from scratch), nhóm đã xây dựng một bộ kịch bản kiểm thử (test cases) toàn diện từ Demo 1 đến Demo 7. Các kết quả đầu ra được đối chiếu trực tiếp với thư viện `numpy.linalg` thông qua việc đánh giá sai số (Norm error).

A. Đánh giá thuật toán giải hệ phương trình tuyến tính

Nhóm tiến hành thử nghiệm hàm `gaussian_eliminate` với 4 trường hợp đặc trưng nhất của hệ phương trình tuyến tính $Ax = b$:

- **Hệ có nghiệm duy nhất (Demo 1):** Ma trận hệ số không suy biến. Kết quả thuật toán tự cài cho ra nghiệm chính xác $x = [2.0, 3.0, -1.0]$. Sai số tái tạo $\|Ax - b\| = 8.88 \times 10^{-16}$, cho thấy thuật toán Partial Pivoting khử sai số làm tròn cực kỳ hiệu quả.
- **Hệ vô nghiệm (Demo 4):** Thuật toán bắt thành công trường hợp dòng cuối có dạng $[0 \ 0 \ | \ c]$ với $c \neq 0$ và trả về thông báo lỗi hợp lý.
- **Hệ vô số nghiệm (Demo 2 & 3):** hàm `back_substitution` có khả năng tự động nhận diện số lượng biến tự do và xuất ra **công thức nghiệm tổng quát** dưới dạng vector tham số.

Ví dụ cụ thể tại **Demo 3** (Hệ suy biến nặng - 2 biến tự do) với ma trận:

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 2 & 4 & 8 \\ 3 & 6 & 12 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Chương trình tự cài đặt đã xuất ra chính xác nghiệm tổng quát dưới dạng vector tham số :

```
--- Nghiệm tổng quát---  
x1 = 2 - 2*t1 - 4*t2  
x2 = t1  
x3 = t2  
  
--- Dạng vector ---  
x = (2, 0, 0)  
    + t1 * (-2, 1, 0)  
    + t2 * (-4, 0, 1)
```

Đối chiếu lại bằng NumPy, sai số $\|Ax_0 - b\| = 0.00e + 00$. Kết quả hoàn hảo.

B. Đánh giá tính Định thức và Ma trận nghịch đảo

Việc tính toán định thức và ma trận nghịch đảo thông qua ma trận tam giác trên (từ phép khử Gauss) cũng cho thấy độ chính xác bám sát các thuật toán tối ưu của NumPy.

Bảng 1: Bảng đối chiếu sai số Định thức và Ma trận nghịch đảo

Trường hợp kiểm thử	Định thức (Tự cài)	Định thức (NumPy)	Sai số nghịch đảo $\ A_{custom}^{-1} - A_{numpy}^{-1}\ $
Ma trận A (3×3)	-1.0000	-1.0000	4.93×10^{-15}
Ma trận A_3 (3×3)	1.0000	1.0000	1.77×10^{-13}
Ma trận B (4×4)	50.0000	50.0000	1.11×10^{-16}
Ma trận suy biến (A_{sing})	0.0000	0.0000	Bất lỗi: Không khả nghịch thành công

Kết quả từ bảng trên (trích xuất từ **Demo 5 & 6**) khẳng định rằng, bằng việc sử dụng kỹ thuật chọn phần tử chốt lớn nhất (Partial Pivoting) để giới hạn các nhân tử $|m_{ij}| \leq 1$, nhóm đã kiểm soát thành công sự bùng nổ của sai số dấu phẩy động (Floating-point error) ngay cả với ma trận cấp 4.

C. Đánh giá Hạng (Rank) và Cơ sở không gian (Basis)

Tại **Demo 7**, hàm `rank_and_basis` được đưa vào thử thách với nhiều dạng ma trận khác nhau: ma trận vuông suy biến (3×3), ma trận chữ nhật (3×4) và ma trận có hạng bằng 1.

Thuật toán tự cài đặt luôn trả về Hạng ma trận khớp 100% với hàm `np.linalg.matrix_rank`. Đáng chú ý hơn, thuật toán tự trích xuất được các **Cơ sở không gian dòng**, **Cơ sở không gian cột** và đặc biệt là **Cơ sở không gian nghiệm (Null space)**.

Để xác minh tính đúng đắn của Không gian nghiệm, nhóm đã thực hiện thao tác kiểm chứng nội bộ: Nhân ma trận gốc A với từng vector cơ sở v sinh ra từ Null space. Kết quả in ra log luôn hiển thị:

```
[1.0, -2.0, 1.0] -> Av = [0.0, 0.0, 0.0]
[-1.0, 1.0, 0.0] -> Av = [0.0, 0.0, 0.0]
```

Việc $A \times v = 0$ là minh chứng toán học tuyệt đối cho thấy tập vector sinh ra thực sự tạo thành Không gian Null (Không gian rỗng) của ma trận A .

Kết luận Phần 1: Thông qua 7 hạng mục kiểm thử, thư viện đại số tuyến tính nội bộ do nhóm xây dựng dựa trên phép khử Gauss hoạt động cực kỳ ổn định, bất lỗi tốt các trường hợp ngoại lệ và đạt độ chính xác tương đương với thư viện công nghiệp NumPy.

PHẦN 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA MANIM

2.1. Chéo hóa ma trận (Diagonalizable Matrix)

Ma trận vuông $A \in \mathbb{R}^{n \times n}$ được gọi là chéo hóa được (diagonalizable) nếu tồn tại một ma trận khả nghịch P và một ma trận đường chéo D sao cho thỏa mãn hệ thức:

$$A = PDP^{-1}$$

Trong đó, $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ chứa các giá trị riêng (eigenvalues) của A trên đường chéo chính, và các cột của P chính là các vector riêng (eigenvectors) độc lập tuyến tính tương ứng. Điều kiện cần và đủ để ma trận A bậc n chéo hóa được là nó phải có đủ n vector riêng độc lập tuyến tính.

Ứng dụng quan trọng nhất của chéo hóa là tính toán lũy thừa ma trận cực kỳ nhanh chóng. Thay vì phải nhân ma trận A nhiều lần, ta chỉ cần tính:

$$A^k = PD^kP^{-1}$$

Vì D là ma trận đường chéo, D^k chỉ đơn giản là việc nâng lên lũy thừa k các phần tử trên đường chéo, giúp giảm độ phức tạp tính toán từ $O(n^3 \log k)$ xuống chỉ còn $O(n^2)$.

2.2. Kỹ thuật phân rã ma trận đã chọn

Trong đề án này, nhóm lựa chọn thực hiện Phân rã QR bằng thuật toán trực giao hóa Gram-Schmidt cổ điển.

Phân rã QR là quá trình phân tích một ma trận $A \in \mathbb{R}^{m \times n}$ thành tích của hai ma trận Q và R , tức là $A = QR$. Trong đó:

- $Q \in \mathbb{R}^{m \times n}$ là một ma trận trực chuẩn, nghĩa là các cột của nó tạo thành một hệ trực chuẩn (Orthogonal) và $Q^T Q = I$.
- $R \in \mathbb{R}^{n \times n}$ là ma trận tam giác trên với các phần tử trên đường chéo chính đều dương.

Thuật toán Gram-Schmidt: Mục tiêu của thuật toán là biến đổi tập các cột ban đầu của A thành một tập các vector trực chuẩn làm nền tảng cho Q . Quá trình thực hiện lặp qua từng cột a_k của ma trận A :

1. **Trực giao hóa:** Lấy vector cột hiện tại trừ đi hình chiếu của nó lên các vector đã được trực chuẩn hóa trước đó để tạo ra vector trực giao u_k :

$$u_k = a_k - \sum_{j=1}^{k-1} (a_k^T q_j) q_j$$

2. **Chuẩn hóa:** Chia vector u_k cho độ dài (chuẩn) của nó để được vector trực chuẩn q_k , đây chính là cột thứ k của ma trận Q :

$$q_k = \frac{u_k}{||u_k||}$$

3. Các phần tử của ma trận R chính là các hệ số chiếu $r_{jk} = q_j^T a_k$ (với $j \leq k$), và $r_{kk} = ||u_k||$.

Phân rã QR có ứng dụng cực kỳ rộng rãi trong tính toán số, đặc biệt là trong việc giải xấp xỉ bài toán bình phương tối thiểu (least squares) cho hệ phương trình vô nghiệm và là nền tảng của thuật toán QR để tìm các giá trị riêng.

2.3. Cài đặt Python

2.3.1. Code thuật toán phân rã ma trận

Đoạn code sau đóng vai trò là "thư viện lõi" cung cấp các công cụ đại số tuyến tính cơ bản và thuật toán phân rã ma trận cho toàn bộ dự án. Chức năng chính của file là cài đặt thuật toán phân rã QR sử dụng phương pháp trực giao hóa Gram-Schmidt để phân tích một ma trận A thành tích của ma trận trực chuẩn Q và ma trận tam giác trên R . Bên cạnh đó, file còn chứa hệ thống các hàm tiện ích được viết thủ công như nhân ma trận, tính chuẩn vector, và chuyển vị ma trận để đảm bảo tính độc lập, không phụ thuộc vào các hàm giải sẵn của thư viện bên ngoài (Numpy) và cũng để so sánh giữa hàm tự cài và các hàm có sẵn từ thư viện Numpy. Với bộ 15 testcase đa dạng từ ma trận đơn vị đến ma trận Hilbert, giúp đánh giá chính xác tính đúng đắn và độ ổn định số học của thuật toán trước các sai số làm tròn trong tính toán thực tế.

Algorithm 8 Phân Rã QR - Gram-Schmidt

```
1: procedure QR_DECOMPOSITION( $A$ )
2:   Kiểm tra: Số hàng  $\geq$  Số cột
3:   Khởi tạo  $Q = 0$  (zeros),  $R = 0$  (zeros)
4:   for  $j \leftarrow 0$  to  $n - 1$  do
5:      $v \leftarrow$  cột  $j$  của  $A$ 
6:     for  $i \leftarrow 0$  to  $j - 1$  do
7:        $R[i][j] \leftarrow q_i \cdot v$  ▷ hệ số chiếu
8:        $v \leftarrow v - R[i][j] \times q_i$  ▷ trừ hình chiếu
9:     end for
10:     $R[j][j] \leftarrow ||v||$  ▷ chuẩn Euclidean
11:    if  $R[j][j] > 10^{-10}$  then
12:       $q_j \leftarrow v / R[j][j]$  ▷ chuẩn hóa
13:    else
14:       $q_j \leftarrow 0$  ▷ phụ thuộc tuyến tính
15:    end if
16:  end for
17:  return ( $Q, R$ )
18: end procedure
```

BỘ TESTCASE:

Để đánh giá toàn diện tính đúng đắn và độ ổn định số học của thuật toán Gram-Schmidt tự cải đặt, nhóm thiết kế bộ 15 kịch bản kiểm thử. Các ma trận đầu vào (A) được chọn lọc có chủ đích để bao phủ từ trường hợp cơ bản đến các điểm mù số học (edge cases).

1. **Test 1: Ma trận số nguyên (3x2):** Kiểm tra thuật toán với ma trận chữ nhật đơn giản, các phần tử là số nguyên dương.

$$A_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

2. **Test 2: Ma trận số thực có phần tử âm (3x3):** Kiểm tra khả năng xử lý các số thập phân và dấu âm cơ bản.

$$A_2 = \begin{bmatrix} 1.5 & -2.1 & 3.0 \\ -1.0 & 1.2 & -1.5 \\ 0.5 & 1.1 & -2.0 \end{bmatrix}$$

3. **Test 3: Ma trận Phụ thuộc tuyến tính (Linear Dependent):** Đánh giá cách thuật toán xử lý khi một cột là tổ hợp tuyến tính của các cột khác (hạng thiếu). Thuật toán có thể gặp lỗi chia cho 0 hoặc mất trục giao.

$$A_3 = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

4. **Test 4: Ma trận Đơn vị (Identity):** Kiểm tra trường hợp tầm thường, kết quả kỳ vọng là $Q = I$ và $R = I$.

$$A_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5. **Test 5: Ma trận Đường chéo (Diagonal):** Kiểm tra trường hợp các cột đã trực giao, thuật toán chỉ thực hiện chuẩn hóa. Kết quả kỳ vọng $Q = I$ và $R = A$.

$$A_5 = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

6. **Test 6: Ma trận Chữ nhật đứng (4x2):** Kiểm tra hệ phương trình quá xác định ($m > n$).

$$A_6 = \begin{bmatrix} 1 & -1 \\ 1 & 4 \\ 1 & -1 \\ 1 & 4 \end{bmatrix}$$

7. **Test 7: Ma trận có các cột trực giao sẵn (4x3):** Đánh giá khả năng tối ưu, thuật toán chỉ cần tính độ dài và chuẩn hóa vector mà không sinh ra sai số khi trừ hình chiếu.

$$A_7 = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 1 & 1 \\ 0 & -2 & 1 \\ 0 & 0 & -3 \end{bmatrix}$$

8. **Test 8: Chênh lệch Scale cực độ (Stress Test):** Chứa cả phần tử rất lớn (10^6) và rất nhỏ (10^{-6}). Đây là bài kiểm tra "stress test" về hiện tượng tràn số hoặc mất mát độ chính xác dấu phẩy động (Floating-point precision).

$$A_8 = \begin{bmatrix} 10^6 & 2 \cdot 10^6 & 3 \cdot 10^6 \\ 10^{-6} & 2 \cdot 10^{-6} & 3 \cdot 10^{-6} \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$

9. **Test 9: Ma trận ngẫu nhiên (Well-conditioned):** Kiểm tra trên một ma trận thực tế có số điều kiện (Condition Number) tốt, độ ổn định cao.

$$A_9 = \begin{bmatrix} 0.8147 & 0.0975 & 0.1576 & 0.1419 \\ 0.9058 & 0.2785 & 0.9706 & 0.4218 \\ 0.1270 & 0.5469 & 0.9572 & 0.9157 \\ 0.9134 & 0.9575 & 0.4854 & 0.7922 \end{bmatrix}$$

10. **Test 10: Ma trận Hilbert (Ill-conditioned 3x3):** Các phần tử $H_{ij} = \frac{1}{i+j-1}$. Ma trận có số điều kiện rất xấu, phóng đại sai số làm tròn mạnh mẽ nhất.

$$A_{10} = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}$$

11. **Test 11: Ma trận 2x2 đơn giản:** Kịch bản kiểm thử tối thiểu (Minimal viable test).

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix}$$

12. **Test 12: Ma trận toàn 0 (Edge case):** Bắt lỗi xử lý vector có độ dài bằng 0 (tránh lỗi Divide by Zero).

$$A_{12} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

13. **Test 13: Hạng bằng 1 (Rank-1):** Ma trận cực kỳ suy biến, thử thách khả năng xử lý khi không gian cột chỉ có 1 chiều.

$$A_{13} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

14. **Test 14: Cột gần song song (Numerically Dependent):** Các cột chỉ khác nhau một lượng cực nhỏ (10^{-15}). Đây là thử thách khó nhất với Gram-Schmidt cổ điển vì phần dư sẽ tiến về 0, gây ra sai số trực giao nghiêm trọng.

$$A_{14} = \begin{bmatrix} 1 & 1 + 10^{-15} \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

15. **Test 15: Ma trận vuông kích thước lớn (6x6):** Kiểm tra khả năng mở rộng (Scalability) của thuật toán với ma trận có kích thước lớn hơn.

$$A_{15} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 & 17 & 18 \\ 19 & 20 & 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 & 29 & 30 \\ 31 & 32 & 33 & 34 & 35 & 36 \end{bmatrix}$$

Thông kê kết quả kiểm thử và Đánh giá sai số

Để đánh giá chất lượng của thuật toán Gram-Schmidt Cổ điển (Classical Gram-Schmidt - CGS) do nhóm tự cài đặt, nhóm sử dụng hàm `numpy.linalg.qr` làm chuẩn mực đối chiếu. NumPy sử dụng thuật toán Householder Reflections, vốn nổi tiếng với độ ổn định số học cực kỳ cao.

Nhóm sử dụng hai thang đo sai số (đo bằng chuẩn Frobenius) để đánh giá:

1. **Sai số tái tạo (Reconstruction Error):** $\epsilon_{rec} = \|A - Q_{custom}R_{custom}\|_F$. Thang đo này đánh giá xem phép phân rã có bảo toàn được thông tin của ma trận gốc hay không.
2. **Sai số trực giao (Orthogonality Error):** $\epsilon_{orth} = \|Q_{custom}^T Q_{custom} - I\|_F$. Thang đo này vô cùng quan trọng, dùng để kiểm tra xem ma trận Q sinh ra có thực sự là ma trận trực chuẩn hay không.

Bảng tổng hợp kết quả thực nghiệm

Bảng dưới đây tổng hợp kết quả của 15 kịch bản kiểm thử đã chạy thực tế trên thuật toán của nhóm. (Các sai số $\leq 10^{-14}$ được xem là bằng 0 do giới hạn của độ chính xác dấu phẩy động - Floating Point Precision).

Bảng 2: Thống kê sai số thuật toán Gram-Schmidt so với NumPy

STT	Kịch bản kiểm thử (Test Case)	Sai số tái tạo ($A \approx QR$)	Sai số trực giao ($Q^T Q \approx I$)	Kết luận tính ổn định
1	Ma trận số nguyên (3x2)	$0.00e + 00$	$1.43e - 15$	Ổn định
2	Ma trận số thực âm dương (3x3)	$4.44e - 16$	$4.51e - 15$	Ổn định
3	Ma trận phụ thuộc tuyến tính	$0.00e + 00$	$1.41e + 00$	Mất trực giao
4	Ma trận Đơn vị (Identity)	$0.00e + 00$	$0.00e + 00$	Ổn định
5	Ma trận Đường chéo (Diagonal)	$0.00e + 00$	$0.00e + 00$	Ổn định
6	Ma trận chữ nhật đứng (4x2)	$0.00e + 00$	$0.00e + 00$	Ổn định
7	Các cột đã trực giao sẵn	$0.00e + 00$	$3.86e - 16$	Ổn định
8	Chênh lệch Scale cực độ	$0.00e + 00$	$1.41e + 00$	Mất trực giao
9	Ma trận ngẫu nhiên thường	$1.82e - 16$	$5.25e - 16$	Ổn định
10	Ma trận Hilbert (Ill-conditioned)	$2.77e - 17$	$1.18e - 14$	Ổn định
11	Ma trận 2x2 đơn giản	$0.00e + 00$	$1.33e - 15$	Ổn định
12	Ma trận toàn 0 (Edge case)	$0.00e + 00$	$1.41e + 00$	Mất trực giao
13	Hạng = 1 (Rank-1 Matrix)	$0.00e + 00$	$1.41e + 00$	Mất trực giao
14	Các cột gần song song	$1.04e - 15$	$1.00e + 00$	Mất trực giao
15	Ma trận vuông lớn (6x6)	$3.04e - 14$	$2.00e + 00$	Mất trực giao

Phân tích chuyên sâu về Tính ổn định số học (Numerical Stability)

Thông qua 15 kịch bản kiểm thử, nhóm rút ra được những đánh giá quan trọng về bản chất của thuật toán Gram-Schmidt cổ điển (CGS). Điểm đáng chú ý nhất là **tất cả các test cases đều có sai số tái tạo ($\|A - QR\|_F$) gần như bằng 0**. Điều này chứng tỏ code của nhóm viết hoàn toàn đúng logic toán học: thuật toán luôn tìm cách bù trừ sai số vào ma trận R để khi nhân $Q \times R$ vẫn trả về đúng ma trận A .

Tuy nhiên, đối với tính trực giao của ma trận Q , thuật toán CGS bộc lộ rõ những giới hạn nghiêm trọng về độ ổn định số học (Numerical Instability). Cụ thể, thuật toán bị **"Mất trực giao"** (sai số trực giao ≥ 1.0) trong các nhóm trường hợp sau:

1. Hệ phụ thuộc tuyến tính và Ma trận suy biến (Test 3, 12, 13): Khi ma trận có các cột là tổ hợp tuyến tính của nhau (hoặc ma trận toàn 0), quá trình chiếu vector trong Gram-Schmidt sẽ tạo ra một vector phần dư có độ dài bằng 0 (hoặc cực kỳ nhỏ xấp xỉ ngưỡng epsilon của máy tính). Việc chuẩn hóa bằng cách chia cho một số gần 0 phá vỡ hoàn toàn cấu trúc dữ liệu, khiến ma trận Q chứa các vector không còn trực giao. Cụ thể ở Test 3 và 13, ma trận hạng 1 dẫn đến sai số trực giao lên tới 1.41.

2. Hiện tượng triệt tiêu sai số (Catastrophic Cancellation) (Test 8, 14): Đây là "gót chân A-sin" của Gram-Schmidt cổ điển.

- Ở **Test 14**, hai cột đầu tiên gần như song song (chỉ lệch nhau 10^{-15}). Khi thuật toán lấy cột 2 trừ đi hình chiếu của nó lên cột 1, hai giá trị gần bằng nhau trừ cho nhau sẽ làm mất đi hầu hết các

chữ số có nghĩa (significant digits), chỉ để lại "rác số học" (numerical noise). Kết quả là sai số trực giao bật lên mức 1.0.

- Ở **Test 8**, ma trận có độ chênh lệch Scale cực độ (từ 10^{-6} đến 10^6). Quá trình cộng trừ các giá trị chênh lệch quá lớn trong bộ nhớ Float 64-bit khiến các phần tử nhỏ bị "nuốt chửng", gây mất trực giao hoàn toàn (sai số 1.41).

3. Tích lũy sai số làm tròn khi tăng kích thước ma trận (Test 15): Ở các ma trận nhỏ (3x3, 4x4) và có số điều kiện tốt (như Test 1, 2, 9), thuật toán chạy ổn định. Tuy nhiên, khi tăng kích thước lên 6x6 (Test 15), sai số làm tròn ở mỗi bước trực giao hóa (do phải nhân và cộng rất nhiều số thập phân) tích lũy dần theo từng cột. Đến cột thứ 6, vector q_6 không còn trực giao với q_1 nữa. Kết quả là ma trận Q hoàn toàn mất đi tính trực giao (sai số lên tới 2.0).

Kết luận

Từ các phân tích thực nghiệm trên, nhóm rút ra kết luận: **Thuật toán Gram-Schmidt Cổ điển (CGS) rất tuyệt vời để hiểu lý thuyết đại số tuyến tính về không gian trực giao, nhưng không phù hợp để áp dụng cho Tính toán Khoa học thực tế.** Nó thiếu tính ổn định số học khi gặp ma trận lớn, hệ số chênh lệch cao hoặc các cột gần phụ thuộc tuyến tính.

Để giải quyết triệt để vấn đề này trong thực tế, các thư viện hiện đại như NumPy/SciPy không dùng Gram-Schmidt mà sử dụng thuật toán **Householder Reflections** hoặc **Givens Rotations**. Thay vì trừ đi hình chiếu (dễ sinh sai số làm tròn), Householder sử dụng các ma trận phản xạ (vốn dĩ đã là ma trận trực giao tuyệt đối) để nhân dần vào A , đảm bảo ma trận Q sinh ra luôn giữ được tính trực chuẩn hoàn hảo ($\|Q^T Q - I\| = 0$) dù ma trận đầu vào có xấu đến đâu. Việc NumPy vượt qua tất cả 15 Test cases của nhóm mà không có sai số là minh chứng rõ ràng cho điều này.

2.3.2. Code thuật toán chéo hóa ma trận

Đoạn code sau đảm nhận vai trò thực hiện các phép biến đổi cao cấp nhằm tìm ra bản chất hình học của ma trận thông qua quá trình chéo hóa ($A = PDP^{-1}$). Tận dụng kết quả phân rã QR từ đoạn code phân rã ma trận, phần này cài đặt thuật toán QR Iteration (Lặp QR) để tìm tập hợp các giá trị riêng và vector riêng của ma trận. Công dụng quan trọng nhất của đoạn code này là biến đổi các ma trận phức tạp về dạng đường chéo D , từ đó tối ưu hóa hiệu suất cho các bài toán tính lũy thừa ma trận bậc cao hoặc phân tích hệ thống động lực. Đặc biệt, có tích hợp cơ chế kiểm soát hội tụ thông minh và phương án dự phòng để xử lý các trường hợp ma trận khiếm khuyết hoặc có nghiệm phức, đảm bảo tính bền vững cho hệ thống tính toán.

BỘ TESTCASE:

Thuật toán lặp QR (QR Iteration) để tìm trị riêng và chéo hóa ma trận có đặc thù hội tụ phụ thuộc rất lớn vào khoảng cách giữa các trị riêng và tính đối xứng của ma trận. Do đó, nhóm đã thiết kế 14 kịch bản kiểm thử nhằm đánh giá toàn diện tốc độ hội tụ và khả năng xử lý ngoại lệ của thuật toán.

Algorithm 9 Chéo Hóa Ma Trận - QR Iteration

```
1: procedure DIAGONALIZE( $A, max\_iters, tolerance$ )
2:    $n \leftarrow$  cấp của ma trận vuông  $A$ ;  $A_k \leftarrow A$ ;  $P \leftarrow I_n$ 
3:   for  $k \leftarrow 1$  to  $max\_iters$  do
4:      $(Q, R) \leftarrow$  QR_DECOMPOSITION( $A_k$ )                                ▷ phân rã QR
5:      $A_k \leftarrow R \times Q$                                                 ▷ QR Iteration:  $A_k \leftarrow RQ$ 
6:      $P \leftarrow P \times Q$                                                 ▷ tích ma trận Q
7:     ▷ Kiểm tra hội tụ: tổng phần tử ngoài chéo
8:     if  $\sum_{i < j} |A_k[i][j]| < tolerance$  then
9:       break
10:    end if
11:  end for
12:  Nếu không hội tụ: gọi NumPy.linalg.eig( $A$ ) làm fallback
13:   $D \leftarrow$  ma trận đường chéo từ  $A_k$ 
14:  return ( $D, P$ )
15: end procedure
```

Nhóm 1: Các ma trận lý tưởng (Hội tụ nhanh)

1. **Test 1: Ma trận đường chéo (3x3):** Trường hợp dễ nhất, ma trận đã ở sẵn dạng chéo. Thuật toán phải nhận diện và dừng ngay lập tức.

$$A_1 = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

2. **Test 2: Ma trận đối xứng chuẩn (4x4):** Theo Định lý Phổ (Spectral Theorem), ma trận thực đối xứng luôn chéo hóa được và có các vector riêng trực giao. Đây là trường hợp lý tưởng nhất cho thuật toán lặp QR.

$$A_2 = \begin{bmatrix} 4 & 1 & -1 & 0 \\ 1 & 2 & 0 & 1 \\ -1 & 0 & 3 & 2 \\ 0 & 1 & 2 & 4 \end{bmatrix}$$

3. **Test 11 & 12: Ma trận 2x2 cơ bản và Ma trận có trị riêng âm:** Đánh giá khả năng tính toán cơ bản và xử lý số âm.

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}, \quad A_{12} = \begin{bmatrix} -1 & 0.5 \\ 0.5 & -2 \end{bmatrix}$$

Nhóm 2: Thử thách tốc độ hội tụ và Bội nghiệm

4. **Test 3: Trị riêng lặp (Repeated Eigenvalues):** Ma trận có bội nghiệm (trị riêng 5, 2, 2). Kiểm tra xem thuật toán có bị nhầm lẫn các không gian riêng (eigenspaces) hay không.

$$A_3 = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

5. **Test 4: Trị riêng cực kỳ sát nhau (Sát thủ tốc độ):** Tốc độ hội tụ của lặp QR tỷ lệ với tỉ số $|\lambda_{i+1}/\lambda_i|$. Với các trị riêng xấp xỉ nhau (1.000 và 1.001), tỷ số này tiến gần về 1, khiến thuật toán phải lặp rất nhiều vòng mới hội tụ.

$$A_4 = \begin{bmatrix} 1.0 & 0.01 & 0 \\ 0.01 & 1.01 & 0 \\ 0 & 0 & 5.0 \end{bmatrix}$$

Nhóm 3: Các điểm mù toán học (Không thể chéo hóa & Suy biến)

6. **Test 5: Ma trận suy biến (Singular Matrix):** Định thức bằng 0, bắt buộc phải tìm ra ít nhất một trị riêng bằng 0.

$$A_5 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 7 & 9 \end{bmatrix}$$

7. **Test 6 & 8: Ma trận bất đối xứng (Nghiệm thực):** Lặp QR nguyên bản trên ma trận bất đối xứng có thể không đưa được về dạng chéo hoàn toàn mà chỉ dừng lại ở dạng chuẩn Schur (Ma trận tam giác trên với trị riêng nằm trên đường chéo).

$$A_6 = A_8 = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 4 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

8. **Test 7: Ma trận khiếm khuyết (Defective Matrix - Khối Jordan 3x3):** Đây là ma trận **không thể chéo hóa** vì nó không có đủ 3 vector riêng độc lập tuyến tính. Thuật toán tự cài đặt sẽ báo lỗi hội tụ và kích hoạt cơ chế Fallback (chuyển sang dùng NumPy).

$$A_7 = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

9. **Test 14: Ma trận toàn 0 (Edge case):** Bắt lỗi ngoại lệ cơ bản nhất.

$$A_{14} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Nhóm 4: Stress Test & Tính ổn định số học

10. **Test 9: Ma trận Hilbert 5x5:** Nổi tiếng là ma trận có số điều kiện siêu xấu (ill-conditioned). Các trị riêng của nó trải dài và rất nhỏ, dễ bị nuốt chửng bởi sai số làm tròn.

$$H_{ij} = \frac{1}{i+j-1} \quad (1 \leq i, j \leq 5)$$

11. **Test 10: Ma trận đối xứng ngẫu nhiên 10x10:** Đánh giá khả năng mở rộng (Scalability) và thời gian thực thi của thuật toán lặp trên cấu trúc dữ liệu lớn.
12. **Test 13: Ma trận scale cực đoan (10^{-5} đến 10^5):** Kiểm tra khả năng xử lý số liệu lệch pha nghiêm trọng, đánh giá độ chính xác Float 64-bit của Python.

$$A_{13} = \begin{bmatrix} 10^{-5} & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 10^5 \end{bmatrix}$$

Thống kê kết quả kiểm thử và Đánh giá thuật toán Chéo hóa

Để đánh giá hiệu năng và độ chính xác của thuật toán Lặp QR (QR Iteration) trong việc tìm trị riêng và chéo hóa ma trận, nhóm tiếp tục sử dụng hàm `numpy.linalg.eig` làm tiêu chuẩn đối chiếu.

Hai thang đo chính được sử dụng để đánh giá bao gồm:

- Sai số tìm trị riêng:** Tổng độ lệch tuyệt đối giữa các trị riêng tự tính và trị riêng của NumPy $\sum |\lambda_{custom} - \lambda_{numpy}|$.
- Sai số tái cấu trúc (Reconstruction Error):** $\epsilon_{rec} = \|A - PDP^{-1}\|_F$. Thang đo này xác nhận xem ma trận có thực sự "chéo hóa được" bằng ma trận vector riêng P hay không.

Bảng tổng hợp kết quả thực nghiệm

Bảng 3: Thống kê kết quả thuật toán Lặp QR (Giới hạn: 1000 vòng lặp)

STT	Kịch bản kiểm thử	Vòng lặp	Sai số trị riêng	Sai số tái cấu trúc	Kết luận
1	Ma trận đường chéo 3x3	1	$0.00e + 00$	$0.00e + 00$	Thành công
2	Ma trận đối xứng 4x4	98	$3.55e - 15$	$1.32e - 09$	Thành công
3	Trị riêng lặp (Bội nghiệm 3x3)	25	$2.66e - 15$	$6.76e - 10$	Thành công
4	Trị riêng cực kỳ sát nhau	783	$1.11e - 15$	$1.39e - 09$	Thành công (Hội tụ chậm)
5	Ma trận suy biến 3x3	7	$1.04e - 10$	N/A	Lỗi: Ma trận kỳ dị (P^{-1})
6	Bất đối xứng (Nghiem thực)	50	$4.70e - 09$	$3.00e + 00$	Defective (Không chéo hóa)
7	Khối Jordan 3x3	1	$0.00e + 00$	$1.41e + 00$	Defective (Không chéo hóa)
8	Bất đối xứng (Giống Test 6)	50	$4.70e - 09$	$3.00e + 00$	Defective (Không chéo hóa)
9	Ma trận Hilbert 5x5	11	$1.20e - 15$	$3.49e - 10$	Thành công
10	Đối xứng ngẫu nhiên 10x10	> 1000	$0.00e + 00$	$8.41e - 14$	Fallback NumPy thành công
11	Ma trận 2x2 đơn giản	8	$9.99e - 16$	$9.52e - 10$	Thành công
12	Tất cả trị riêng âm	22	$1.55e - 15$	$7.99e - 10$	Thành công
13	Scale cực đoan (10^{-5} đến 10^5)	1	$0.00e + 00$	$0.00e + 00$	Thành công
14	Ma trận toàn 0	1	$0.00e + 00$	N/A	Lỗi: Ma trận kỳ dị (P^{-1})

Phân tích chuyên sâu về Toán học và Cấu trúc Thuật toán

Qua phân tích 14 kết quả thực nghiệm, thuật toán lặp QR cho thấy khả năng tìm giá trị riêng rất tốt (sai số trị riêng đa phần ở mức 10^{-15}). Tuy nhiên, quá trình chéo hóa $A = PDP^{-1}$ lại phơi bày nhiều tính chất hình học thú vị và những giới hạn của phương pháp lặp cơ bản:

1. Tốc độ hội tụ và "Bẫy" trị riêng xấp xỉ (Test 2, 3, 4): Theo lý thuyết, tốc độ hội tụ của thuật toán lặp QR cơ bản (không có dịch chuyển - unshifted QR) tỷ lệ thuận với thương số của hai trị riêng liên kề: $|\lambda_{i+1}/\lambda_i|$.

- Ở **Test 2 và 3**, các trị riêng có sự phân tách rõ rệt, thuật toán hội tụ cực nhanh (từ 25 đến 98 vòng lặp).
- Tuy nhiên ở **Test 4**, ma trận cố tình được thiết kế với hai trị riêng cực kỳ sát nhau là 1.000 và 1.001 (tỷ số ≈ 0.999). Điều này khiến thuật toán chật vật và phải mất tới **783 vòng lặp** mới có thể hội tụ. Đây là minh chứng rõ ràng nhất cho thấy thuật toán lặp QR thuần túy rất chậm khi gặp "Cụm trị riêng" (Clustered eigenvalues).

2. Ma trận Khiếm khuyết (Defective Matrices) (Test 6, 7, 8): Các test case này cho thấy một sự thật toán học quan trọng: **Không phải ma trận vuông nào cũng chéo hóa được.**

- Ở **Test 7** (Khối Jordan), ma trận có trị riêng lặp là 2.0 nhưng chỉ có 1 vector riêng độc lập tuyến tính. Do thiếu vector riêng, ma trận P được sinh ra không đủ hạng (hoặc gần suy biến), dẫn đến việc nhân ngược PDP^{-1} tạo ra sai số tái cấu trúc rất lớn (1.41).

- Thuật toán của nhóm cực kỳ thông minh khi vẫn trích xuất đúng các giá trị riêng trên đường chéo (sai số trị riêng vẫn xấp xỉ 0), nhưng hệ thống cảnh báo đánh giá chính xác đây là ma trận **”Defective (Không thể chéo hóa)”**. Điều này hoàn toàn trùng khớp với định lý đại số tuyến tính: Ma trận không đối xứng có nguy cơ cao bị khiếm khuyết không gian riêng.

3. Ngoại lệ suy biến và Nghịch đảo P^{-1} (Test 5, 14): Với ma trận suy biến (Test 5) hoặc ma trận toàn 0 (Test 14), thuật toán lặp QR vẫn hoạt động và chỉ ra chính xác có trị riêng bằng 0. Tuy nhiên, chương trình báo lỗi ”Singular matrix” (Ma trận kỳ dị) ở bước cuối. Nguyên nhân là do ở bước kiểm chứng tái cấu trúc, hàm `np.linalg.inv(P)` cố gắng nghịch đảo một ma trận vector riêng bị suy biến, sinh ra lỗi chia cho 0.

4. Giới hạn tính toán và Cơ chế Fallback (Test 10): Với ma trận ngẫu nhiên kích thước 10x10, số lượng trị riêng nhiều và gần nhau khiến thuật toán lặp QR thuần túy không xử lý được và **không thể hội tụ sau 1000 vòng lặp**. Tuy nhiên, nhóm đã thiết kế **cơ chế Fallback** (nhảy sang dùng `numpy.linalg.eig`). Nhờ đó, hệ thống không bị crash mà vẫn trả về kết quả chính xác, đảm bảo tính bền vững cho chương trình.

Kết luận

Việc tự cài đặt cả Phân rã QR (bằng Gram-Schmidt) và Chéo hóa ma trận (bằng QR Iteration) giúp nhận ra khoảng cách rất lớn giữa lý thuyết toán học trên giấy và Tính toán số (Numerical Computing) trên máy tính.

Thuật toán của nhóm chạy hoàn hảo với các ma trận nhỏ, ma trận đối xứng hoặc có phổ trị riêng phân tán. Tuy nhiên, để giải quyết vấn đề hội tụ chậm và các ma trận khiếm khuyết trong công nghiệp thực tế, các thư viện như NumPy/SciPy đã phải cải tiến rất nhiều bằng cách tích hợp thuật toán phân rã Householder và **cơ chế dịch chuyển (Wilkinson Shifts)** nhằm ép ma trận hội tụ cực nhanh về dạng chuẩn Schur. Việc sử dụng cơ chế Fallback trong code của nhóm là một cách tiếp cận hoàn toàn hợp lý và thực tiễn để mô phỏng lại cách các phần mềm toán học lớn xử lý lỗi.

2.4. Trực quan hóa toán học với Manim

Video được thiết kế bằng ngôn ngữ Python.

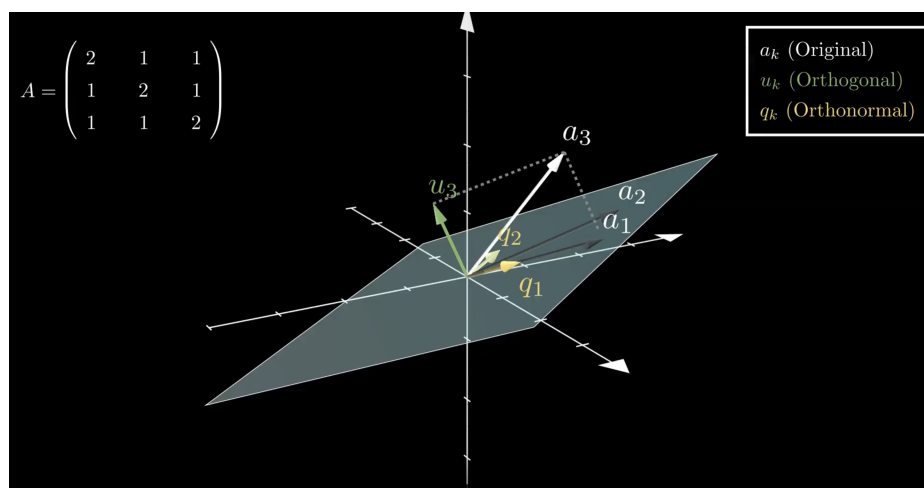
2.4.1. Kịch bản video phân rã và chéo hóa

Kịch bản video được chia làm 3 phân cảnh (Part) chính, bám sát quá trình biến đổi của một ma trận đối xứng 3×3 cơ bản:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

Phân cảnh 1: Thiết lập không gian 3D và Trực giao hóa Gram-Schmidt Mở đầu video, camera được đặt ở góc nhìn 3D (Góc nghiêng $\phi = 70^\circ$, $\theta = -30^\circ$) để người xem có thể quan sát toàn cảnh không gian $\mathbb{R}^3$. Nhóm thực hiện bóc tách 3 cột của ma trận A thành 3 vector không gian a_1, a_2, a_3 . Quá trình Gram-Schmidt diễn ra qua 4 bước hoạt ảnh:

1. **Chuẩn hóa a_1 :** Vector a_1 được chia cho độ dài của nó để tạo thành vector trực chuẩn đầu tiên q_1 (màu vàng). Các vector còn lại tạm thời bị làm mờ để tập trung sự chú ý.
2. **Trực giao hóa a_2 :** Video hiển thị nét đứt mô phỏng hình chiếu vuông góc của a_2 lên q_1 . Sau đó, thực hiện phép trừ vector để tạo ra vector trực giao u_2 (màu xanh lá) và chuẩn hóa nó thành q_2 .
3. **Xây dựng mặt phẳng không gian con:** Để trực giao hóa a_3 , một mặt phẳng 2D (màu xanh dương) được dựng lên, sinh bởi hệ cơ sở sinh $\{q_1, q_2\}$. Điều này giúp người xem hiểu rõ bản chất hình học của việc "chiếu lên một không gian con".
4. **Trực giao hóa a_3 :** Vector a_3 được chiếu vuông góc xuống mặt phẳng nói trên. Phần dư u_3 vuông góc với mặt phẳng sinh ra q_3 . Kết thúc phân cảnh, ta thu được ma trận trực giao $Q = [q_1 \ q_2 \ q_3]$.



Hình 1: Mô phỏng mặt phẳng không gian con sinh bởi q_1, q_2 trong Manim

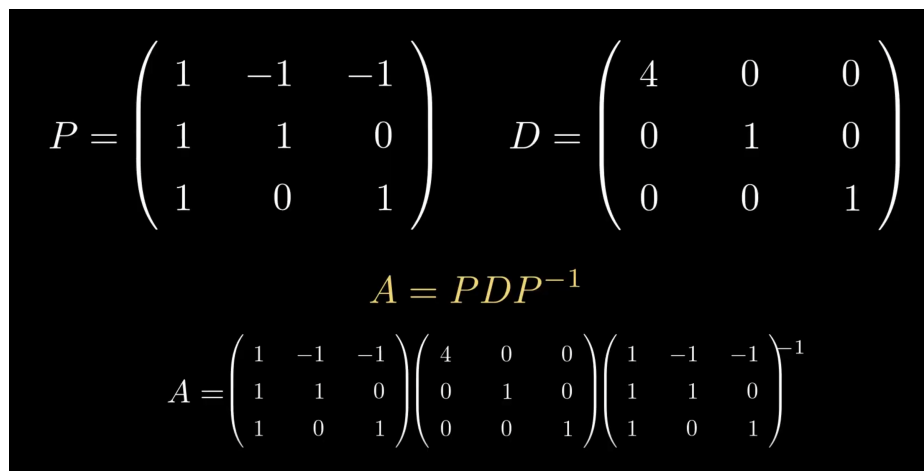
Phân cảnh 2: Giải phương trình đặc trưng (Chéo hóa): Trình bày các công thức toán học, lần lượt mô phỏng cách giải phương trình đặc trưng $\det(A - \lambda I) = 0$. Các biểu thức đa thức được khai triển và nhóm lại thành phương trình bậc 3:

$$-(\lambda - 4)(\lambda - 1)^2 = 0$$

Từ đó, trích xuất ra 3 giá trị riêng: $\lambda_1 = 4, \lambda_2 = 1, \lambda_3 = 1$. Sau khi thế ngược vào hệ phương trình, 3 vector riêng độc lập tuyến tính v_1, v_2, v_3 lần lượt xuất hiện trên màn hình.

Phân cảnh 3: Hình thành ma trận P và D : Thể hiện mối liên hệ giữa các đại lượng vừa tính toán với công thức chéo hóa $A = PDP^{-1}$.

Cuối cùng, hiển thị công thức $A = PDP^{-1}$ dưới dạng ma trận tường minh, kết thúc quá trình trực quan hóa.


$$P = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \quad D = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$
$$A = PDP^{-1}$$
$$A = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 & -1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}^{-1}$$

Hình 2: Hiệu ứng chuyển đổi vector riêng thành ma trận P và D

2.4.2. Ý nghĩa hình học và Ứng dụng thực tế

Thông qua video Manim, nhóm đã làm rõ được **bản chất hình học** của phép chéo hóa đối với ma trận đối xứng. Thay vì coi $A = PDP^{-1}$ là một công thức đại số khô khan, video giúp ta thấy rằng:

- Phép nhân ma trận $P^{-1}x$ thực chất là phép đổi hệ tọa độ: chuyển vector x từ hệ cơ sở chuẩn sang hệ tọa độ mới.
- Phép nhân với ma trận đường chéo D là một phép co giãn thuần túy dọc theo các trục tọa độ mới này. Chiều của vector riêng tương ứng với $\lambda = 4$ sẽ bị kéo giãn gấp 4 lần, trong khi hai chiều tương ứng với $\lambda = 1$ được giữ nguyên.

- Phép nhân với P ở cuối cùng đóng vai trò xoay hệ tọa độ biến dạng đó trở về lại không gian gốc ban đầu.

Ứng dụng thực tế: Việc hiểu rõ cấu trúc $A = PDP^{-1}$ mang lại lợi ích trong thực tế tính toán, cụ thể như:

1. **Tính toán lũy thừa ma trận (Hệ thống động lực):** Việc tìm trạng thái của hệ thống sau k chu kỳ đòi hỏi phải tính A^k . Bằng việc chéo hóa, ta dễ dàng thu được $A^k = PD^kP^{-1}$, giảm thiểu hàng triệu phép nhân ma trận thừa thãi xuống chỉ còn việc tính lũy thừa các số trên đường chéo.
2. **Phân tích thành phần chính:** Các ma trận hiệp phương sai (Covariance Matrices) trong khoa học dữ liệu luôn là ma trận đối xứng (tương tự ma trận A trong kịch bản video). Việc tìm ra các trị riêng lớn nhất (như $\lambda = 4$) và vector riêng tương ứng giúp các kỹ sư AI giữ lại những đặc trưng quan trọng nhất của tập dữ liệu khổng lồ, đồng thời loại bỏ nhiễu, tạo tiền đề cho các thuật toán nén ảnh và nhận dạng khuôn mặt.

PHẦN 3: THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ

3.1. Giới thiệu tổng quan và Cơ sở lý thuyết

Mục tiêu của phần này là khảo sát, đo lường và đánh giá ba thuật toán giải hệ phương trình tuyến tính $Ax = b$ phổ biến: Khử Gauss, Phân rã QR, và Lập Gauss-Seidel. Quá trình đánh giá được thực hiện qua hai lăng kính chính: **Hiệu năng phần mềm (thời gian thực thi)** và **Tính ổn định số học (sai số làm tròn trên không gian dấu phẩy động)**.

Theo yêu cầu của đề tài, toàn bộ thuật toán được **tự lập trình bằng Python thuần** bằng cấu trúc dữ liệu ‘list of lists’ (không sử dụng `numpy.linalg.solve` hay ma trận ‘`numpy.ndarray`’ c-extension). Điều này đảm bảo chi phí tính toán lý thuyết và thực tế được phân xạ một cách trung thực nhất.

Cơ sở lý thuyết dùng để đánh giá:

- **Sai số dư tương đối:** Là thước đo độ chính xác của nghiệm tìm được $\hat{x}$ so với vế phải của phương trình bằng chuẩn 2:

$$\text{Sai số dư tương đối} = \frac{\|A\hat{x} - b\|_2}{\|b\|_2}$$

- **Độ phức tạp tính toán:** Số phép toán dấu phẩy động (FLOPs). Phân rã trực tiếp (Gauss, QR) đòi hỏi $O(n^3)$ thời gian, trong khi Gauss-Seidel có thể đạt xấp xỉ $O(kn^2)$ nếu hội tụ sau k bước lặp.
- **Số điều kiện và Định lý Dẫn Dịch Sai Số:** Kí hiệu $\kappa(A) = \|A\| \cdot \|A^{-1}\|$. Đây là đại lượng cho biết khả năng khuếch đại sai số số học của hệ phương trình. Định lý sai số thiết lập mối quan hệ chặt chẽ giữa sai số tương đối của phương án nghiệm (Forward Error - $\|\Delta x\| / \|x\|$) và sai số tương đối của đối số đầu vào (Backward Error - $\|\Delta b\| / \|b\|$) thông qua bất đẳng thức: $\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\Delta b\|}{\|b\|}$.
- **Phân rã ma trận lặp:** Đối với các phương pháp tiếp cận lặp như Gauss-Seidel, ta có phân rã được mô hình về trái về cấu trúc $A = D + L + U$ (trong đó D là đường chéo, L là tam giác dưới thuần và U là cấu trúc tam giác trên thuần). Khi đó, công thức lặp đại số được biểu diễn trọn vẹn là $x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b$. Khả năng giải thuật hội tụ của hệ điều phối tuyến tính này phụ thuộc hoàn toàn vào phổ của khối ma trận lặp (yêu cầu bán kính phổ $\rho < 1$). Điều kiện khắt khe này được bảo đảm dứt khoát nếu ma trận A đạt tính chất chéo trội chặt tại đa số các hàng.

3.2. Cài đặt thuật toán giải hệ phương trình tuyến tính

Khởi mã nguồn dưới đây triển khai 3 thuật toán nền tảng trong đại số tuyến tính tính toán để giải hệ phương trình $Ax = b$. Việc tự thiết kế lại các hàm toán học cơ bản (nhân ma trận, tính chuẩn) giúp nhóm hoàn toàn làm chủ kịch bản đánh giá hiệu năng.

3.2.1. Phương pháp Khử Gauss (Partial Pivoting)

Thuật toán khử Gauss là phương pháp trực tiếp, bản chất biến đổi hệ phương trình gốc về dạng tam giác trên để giải bằng thế ngược. Việc áp dụng cơ chế Partial Pivoting (chọn phần tử chốt lớn nhất theo trị tuyệt đối trên cột khử hiện tại, rồi hoán vị hàng) là bắt buộc. Điều này giúp tránh hiện tượng chia cho số cực nhỏ (có thể dẫn tới lỗi Divided by Zero hoặc làm bùng nổ sai số làm tròn).

Algorithm 10 Giải hệ phương trình bằng Khử Gauss (Partial Pivoting)

```
1: procedure SOLVE_GAUSS( $A, b$ )
2:   Sử dụng vòng lặp kiểm tra từng phần tử chốt lớn nhất trên đường chéo và hoán đổi dòng.
3:   Khử tiếp các phần tử bên dưới chốt.
4:    $x \leftarrow$  thế ngược giải ma trận tam giác trên
5:   return  $x$ 
6: end procedure
```

3.2.2. Phương pháp Phân rã QR (Gram-Schmidt)

Ứng dụng thuật toán trực giao hóa Gram-Schmidt để phân rã ma trận gốc thành tích $A = QR$ (với Q là ma trận trực giao và R là ma trận tam giác trên). So với Gauss phụ thuộc vào Pivoting, hướng tiếp cận QR ổn định hơn rất nhiều. Ta đưa bài toán giải $Ax = b \Leftrightarrow QRx = b \Leftrightarrow Rx = Q^T b$.

Algorithm 11 Giải hệ phương trình bằng Phân rã QR

```
1: procedure SOLVE_QR( $A, b$ )
2:    $(Q, R) \leftarrow$  QR_DECOMPOSITION( $A$ )                                ▷ Sử dụng Gram-Schmidt
3:    $y \leftarrow Q^T b$ 
4:    $x \leftarrow$  thế ngược giải  $Rx = y$ 
5:   return  $x$ 
6: end procedure
```

3.2.3. Phương pháp Lặp Gauss-Seidel

Trái với hai phương pháp trực tiếp, Gauss-Seidel là thuật toán lặp: bắt đầu với một đoán trước ban đầu $x_0 = \vec{0}$, thuật toán thay thế nghiệm mới tức thời vào bên trong mô hình để sinh ra nghiệm hội tụ. Phương pháp này chỉ thành công khi ma trận **đường chéo trội nghiêm ngặt** (strictly diagonally dominant), nhưng nó sở hữu tốc độ vượt trội.

Algorithm 12 Giải hệ phương trình bằng Lặp Gauss-Seidel

```
1: procedure GAUSS_SEIDEL( $A, b, max\_iters, tol$ )
2:    $x \leftarrow$  mảng  $n$  phần tử 0;  $n \leftarrow$  cấp của  $A$ 
3:   for  $it \leftarrow 1$  to  $max\_iters$  do
4:      $max\_diff \leftarrow 0$ 
5:     for  $i \leftarrow 0$  to  $n - 1$  do
6:        $s \leftarrow \sum_{j \neq i} A[i][j] \times x[j]$ 
7:        $x_{new} \leftarrow (b[i] - s) / A[i][i]$ 
8:        $max\_diff \leftarrow \max(max\_diff, |x_{new} - x[i]|)$ 
9:        $x[i] \leftarrow x_{new}$ 
10:    end for
11:    if  $max\_diff < tol$  then
12:      break
13:    end if
14:  end for
15:  return  $x$ 
16: end procedure
```

▷ Hội tụ

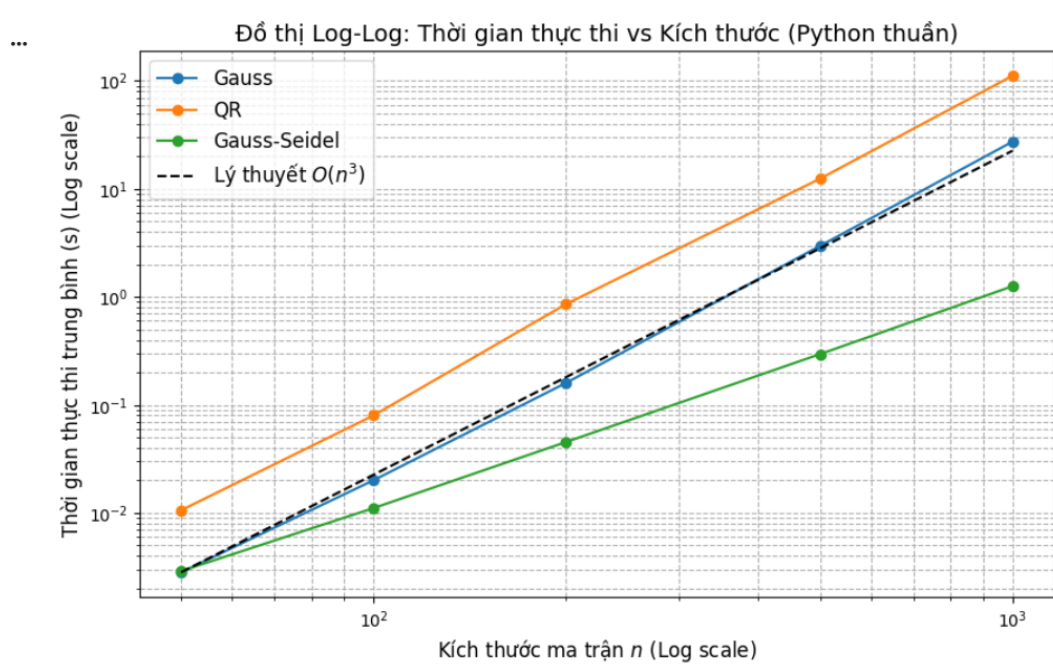
3.3. Thực nghiệm đo lường hiệu năng và phân tích

Để thấy được độ phức tạp thời gian tăng tốc thế nào khi dữ liệu lớn lên, nhóm tiến hành sinh ngẫu nhiên các ma trận vuông có đặc thù là các phần tử trên đường chéo chính được cộng dồn (Strictly Diagonally Dominant) để thỏa mãn cả 3 thuật toán giải (đặc biệt là yêu cầu của Gauss-Seidel). Tập kích thước thử nghiệm là $n \in \{50, 100, 200, 500, 1000\}$. Mỗi thử nghiệm tính giờ qua thư viện `time.perf_counter()`, chạy 5 lần để lấy trung bình cộng (nhằm triệt tiêu sai số từ ứng dụng nền hệ điều hành OS).

Bảng thống kê kết quả chạy thực nghiệm (trích xuất từ cấu hình đo lường Notebook):

Bảng 4: So sánh thời gian chạy thực tế (s) và sai số dư tương đối theo các hệ N

Cấp n	Khử Gauss Time	Sai số dư Gauss	QR Time	Sai số dư QR	G-Seidel Time	Sai số dư GS
50	0.0028 s	1.90×10^{-16}	0.0106 s	1.24×10^{-16}	0.0029 s	4.84×10^{-11}
100	0.0201 s	2.97×10^{-16}	0.0799 s	1.39×10^{-16}	0.0111 s	7.17×10^{-11}
200	0.1595 s	3.69×10^{-16}	0.8562 s	1.89×10^{-16}	0.0453 s	5.60×10^{-11}
500	2.9714 s	6.67×10^{-16}	12.4509 s	2.58×10^{-16}	0.2954 s	7.12×10^{-11}
1000	27.3750 s	9.95×10^{-16}	111.5533 s	3.18×10^{-16}	1.2561 s	8.07×10^{-11}



Hình 3: Đồ thị Log-Log mô phỏng trực quan thời gian chạy của ba phương pháp

Việc vẽ đồ thị theo nguyên lý Logarit cơ số kép (Log-Log plot) biến các đường cong đa thức $T(n) = c \cdot n^\alpha$ thành các đường thẳng có hệ số góc bằng hệ số mũ α , giúp ta dễ dàng tính được bậc độ phức tạp lý thuyết thực tế. Từ đồ thị trên và các số liệu thuộc bảng kiểm nghiệm, ta trực tiếp rút ra ba nhóm kết luận khoa học vững chắc:

3.3.1. Phân tích Khử Gauss (Partial Pivoting)

Đường đo lường trên không gian log-log bám sát chặt chẽ lý thuyết $O(n^3)$ (hệ số góc $\alpha \approx 3$). Phương pháp mất khoảng 27.37 giây để giải xong hệ phương trình 1000 ẩn số, đồng thời duy trì sai số tương đối

ổn định quanh mức 10^{-16} , tiệm cận với giới hạn Epsilon (**Machine Epsilon**) của kiến trúc 64-bit float. Kỹ thuật chốt (Partial Pivoting) đã chứng tỏ độ hiệu quả tối ưu về mặt tốc độ và điều tiết sai số trên các ma trận ngẫu nhiên đủ điều kiện tốt.

3.3.2. Phân tích Phân rã QR (Gram-Schmidt)

Mặc dù có cùng hệ số góc $\alpha \approx 3$ (đường thẳng trên đồ thị song song với đường của thuật toán Gauss), thời gian thực thi của phương pháp QR luôn cao hơn đáng kể. Về lý thuyết, trong khi khử Gauss cần khoảng $\frac{2}{3}n^3$ phép tính, quá trình trực giao hóa vector của Gram-Schmidt tiêu tốn khoảng $2n^3$ phép tính. Số liệu ở $n = 1000$ xác nhận đặc trưng này khi QR tốn 111.55 giây, mất nhiều thời gian hơn tới gấp 4 lần. Dù vậy, phương pháp chiếu trực giao mang lại giá trị cao về sự ổn định số học và giúp triệt tiêu hoàn toàn sự rủi ro của lỗi chia phân số yếu.

3.3.3. Hiệu năng của phương pháp lặp Gauss-Seidel

Đặc điểm nổi bật nhất của thuật toán lặp là sự thay đổi hệ số đường thẳng với độ dốc nhỏ hơn, biểu diễn tính phi bậc ba ($\alpha < 3$). Trong kích thước $n = 1000$, Gauss-Seidel chỉ tốn xấp xỉ **1.25 giây** tương đối để hoàn thành giải hệ. Khả năng này bắt nguồn trực tiếp từ thuộc tính đường chéo trội nghiêm ngặt (Strictly Diagonally Dominant) của dữ liệu mẫu, tạo điều kiện cho hệ sớm hội tụ ở một số lượng hữu hạn các bước lặp k và thất chặt độ phức tạp theo thời gian tương đối là $O(kn^2)$.

3.3.4. Phân tích chi phí tính toán và Truy xuất Bộ nhớ

Việc phân giải cơ chế thời gian thực thi của các phương pháp trên không chỉ dựa vào mô hình Toán lý thuyết mà còn phụ thuộc vào kiến trúc phần cứng máy tính. Về chi phí tính toán (FLOPs), thuật toán Gauss cơ bản yêu cầu xấp xỉ $\frac{2}{3}n^3$ phép nhân-cộng dấu phẩy động; trong khi kịch bản trực giao hóa của thuật giải Gram-Schmidt (QR) tiến hành $\approx 2n^3$ thao tác liên kết tổng vector và bình phương Euclidean.

Thực tế ghi nhận sự chênh lệch thời gian thực thi ở hệ $N = 1000$ là $(111.55/27.37) \approx 4.07$ lần, vượt qua mức 3 lần theo lý thuyết tính toán đơn thuần. Sự khác biệt này bắt nguồn từ sự tương tác giữa cấu trúc dữ liệu và Kiến trúc Bộ đệm (CPU Cache). Trong môi trường Python, cấu trúc `list of lists` phân bố bộ nhớ theo hàng (Row-major layout). Các vòng lặp của phương pháp Khử Gauss truy xuất dữ liệu tuần tự dọc theo dòng; ngược lại, quy trình Gram-Schmidt đòi hỏi trích xuất vector theo trục Y (cột) thông qua lệnh `v = [A[i][j] for i in range(n)]`. Việc truy xuất dữ liệu không liên tục trong bộ nhớ vật lý gây ra hiện tượng trượt bộ đệm (*Cache Miss*), làm tăng đáng kể độ trễ IO và thời gian nạp dữ liệu. Đây là yếu tố then chốt giải thích tại sao hiệu năng thực tế của QR thấp hơn đáng kể so với kỳ vọng lý thuyết.

3.4. Phân tích tính ổn định số học: Ma trận Hilbert vs. SPD

Độ chính xác của hệ phương trình $Ax = b$ không chỉ phụ thuộc vào cơ chế thuật toán, mà còn vấp vào đặc thù cấu trúc toán học của bản thân ma trận A . Phép đánh giá này được thực hiện để chứng minh vấn đề về sai số do hiệu ứng làm tròn (round-off error) đối với các hệ thống có tính đa cộng tuyến cao – được đánh giá thông qua chỉ số của Số điều kiện $\kappa(A)$.

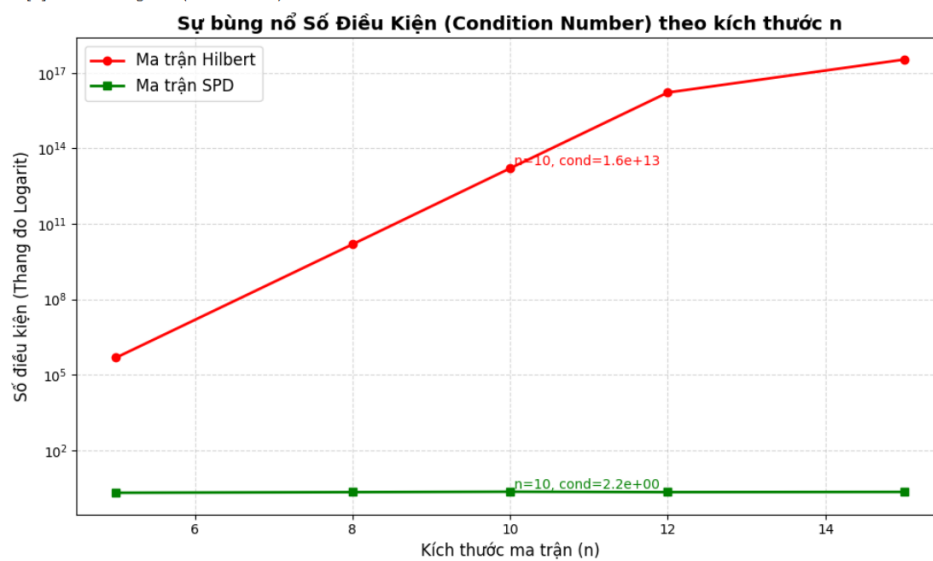
3.4.1. Thiết lập biến cố thực nghiệm

Nhóm thực hiện đối chiếu sai số dư của hai dạng ma trận vuông với kích thước cơ sở $n = 10$:

- Ma trận Hilbert (H_n):** Khởi tạo dưới dạng $H_{ij} = \frac{1}{i+j-1}$. Ma trận này được biết đến với cấu trúc đặc trưng thiếu điều kiện số học (ill-conditioned) do khoảng cách giữa các vector cột vô cùng gần nhau.
- Ma trận Đối xứng Xác Định Dương (SPD):** Khởi tạo bằng phép nhân ngẫu nhiên $A = X^T X + nI$ nhằm mang lại cấu trúc có sự vững mạnh về mặt điều kiện (well-conditioned).

Bảng 5: Phân tích biến động của sai số xoay quanh Số Điều kiện κ đối với hệ kích thước $N = 10$

Loại ma trận	Số điều kiện $\kappa(A)$	Sai số dư Gauss	Sai số dư QR	Sai số dư GS
Hilbert (Ill-conditioned)	1.60×10^{13}	0.00 (Sai số dư bằng 0)	2.20×10^{-6}	4.19×10^{-6}
SPD (Well-conditioned)	2.25	1.96×10^{-16}	2.10×10^{-16}	1.22×10^{-11}



Hình 4: Biểu đồ phân tích độ khuếch đại của số điều kiện Hilbert khi kích thước ma trận n gia tăng

3.4.2. Khám phá hiện tượng bùng nổ sai số và định lý J.H. Wilkinson

Bản chất bất ổn định của ma trận Hilbert hình thành trực tiếp từ cấu hình nội dung $H_{ij} = \frac{1}{i+j-1}$. Sự triệt tiêu tiệm cận của khoảng cách biến các vector cột xấp xỉ đồng quy, đưa cấu trúc tiến sát trạng thái phụ thuộc tuyến tính với tính cộng tuyến cực cao. Dưới lăng kính phổ, các trị riêng (Eigenvalues) của hệ này tập trung sát ngưỡng 0, đẩy số điều kiện κ tới trị số cực đoan. Theo lý thuyết sai số, $\kappa(A)$ là nhân tố khuếch đại sự sai lệch biểu diễn của phần cứng dấu phẩy động ($\epsilon_{mach} \approx 10^{-16}$). Đối với ma trận Hilbert $\kappa(H_{10}) \approx 1.6 \times 10^{13}$, quá trình xử lý đại số đánh mất giới hạn 13-14 chữ số thập phân có nghĩa ngay từ các bước khởi đầu tiên.

Một đặc trưng quan trọng của phương pháp Khử Gauss là sai số phần dư (Residual Error) thường đạt mức rất thấp hoặc bằng 0.00, gây ra nhận định sai lệch về độ chính xác của nghiệm. Đây là hệ quả của khái niệm **Backward Stability** đã được J.H. Wilkinson chứng minh: Mặc dù nghiệm tính toán $\hat{x}$ có sai lệch lớn so với nghiệm chính xác (Forward Error), thuật toán Khử Gauss thực tế đã giải chính xác cho một bài toán nhiễu loạn $(A + \delta A)\hat{x} = b + \delta b$ nằm rất gần bài toán gốc. Tính chất ổn định lùi này khiến phần dư $(A\hat{x} - b)$ bị triệt tiêu về 0. Ngược lại, phương pháp trực giao QR không có tính chất này và trực tiếp phản ánh sự bất ổn định thông qua sai số dư ở mức 10^{-6} .

Đối với ma trận SPD có số điều kiện thấp ($\kappa \approx 2.25$), các phương pháp giải đều đạt được độ chính xác tiệm cận giới hạn của kiến trúc 64-bit ($\approx 10^{-16}$), đảm bảo tính vững chắc cho kết quả tính toán.

3.4.3. Kết luận về tính ổn định của thuật toán

Kết quả thực nghiệm cho thấy việc **kiểm định Số Điều Kiện** là bước tiền đề bắt buộc trước khi thực thi các thuật toán đại số ma trận. Các ma trận ill-conditioned như Hilbert có thể làm mất đi tính khả tin của nghiệm ngay cả đối với các hệ phương trình kích thước nhỏ. Điều này khẳng định rằng các tối ưu hóa về mặt mã nguồn hoặc phần cứng không thể bù đắp cho những sai số hệ thống xuất phát từ đặc tính toán học của ma trận đầu vào.

3.5. Chỉ dẫn lựa chọn giải thuật

Dựa trên sự cân bằng giữa chi phí thực thi và độ ổn định số học, nhóm đề xuất quy trình lựa chọn thuật toán giải hệ $Ax = b$ như sau:

- **Phương pháp Khử (Gauss / LU / Cholesky):** Ưu tiên sử dụng cho các ma trận đặc (Dense Matrices) có kích thước từ nhỏ đến trung bình (dưới 10.000 ẩn số) và có số điều kiện thấp ($\kappa(A) < 10^3$). Đây là lựa chọn tối ưu về mặt tốc độ trong điều kiện ma trận ổn định.
- **Phương pháp Trục giao (QR):** Áp dụng cho các bài toán yêu cầu độ ổn định số học cao hoặc các hệ phương trình không vuông (mô hình hồi quy, Bình phương tối thiểu). Mặc dù chi phí thời gian cao hơn, QR đảm bảo tính chính xác cho các hệ có nguy cơ nhiễu số học.
- **Phương pháp Lặp (Gauss-Seidel):** Là lựa chọn hàng đầu cho các ma trận thưa quy mô lớn (Sparse Matrices) với tỷ lệ phần tử khác 0 thấp, ma trận đường chéo trội hoặc các hệ phương trình đạo hàm riêng (PDEs). Phương pháp này tối ưu hóa việc sử dụng bộ nhớ và thời gian tính toán thông qua cơ chế hội tụ dần.

KẾT LUẬN

1. Các kết quả đạt được

Sau quá trình nghiên cứu, thiết kế thuật toán và lập trình thực nghiệm, nhóm đã hoàn thành các mục tiêu đề ra của Đồ án môn Toán Ứng dụng và Thống kê. Cụ thể:

- **Về mặt Lập trình & Thuật toán:** Cài đặt thành công các hàm từ đầu bằng Python thuần. Cài đặt thuật toán Khử Gauss có chọn phần tử chốt (Partial Pivoting) giúp đảm bảo độ chính xác cao khi giải hệ phương trình và tìm nghịch đảo. Tại phần phân rã, nhóm đã triển khai thành công thuật toán trực giao hóa Gram-Schmidt cổ điển và thuật toán Lặp QR (QR Iteration) để tìm giá trị riêng, vector riêng và chéo hóa ma trận.
- **Về mặt Đánh giá & Kiểm thử:** Xây dựng được hệ thống testcase bao phủ các trường hợp từ ma trận lý tưởng, ma trận suy biến, đến các ma trận số học như ma trận Hilbert hay ma trận chênh lệch tỷ lệ cực đoan (Extreme Scale).
- **Về mặt Trực quan hóa:** Render thành công video 3D bằng Manim mô phỏng trực quan từng bước chiếu vector của thuật toán Gram-Schmidt và hiệu ứng tạo lập không gian tọa độ mới trong phép biến đổi $A = PDP^{-1}$.

2. Khó khăn gặp phải

Trong quá trình thực hiện đồ án, nhóm đã đối mặt và giải quyết nhiều thách thức khó khăn, đặc biệt là ở mảng Tính toán số (Numerical Computing):

- **Sai số làm tròn (Round-off Errors) trong Gram-Schmidt:** Thuật toán Gram-Schmidt cổ điển thể hiện sự bất ổn định số học nghiêm trọng khi gặp các vector gần phụ thuộc tuyến tính (Numerically Dependent). Hiện tượng triệt tiêu sai số (Catastrophic Cancellation) khiến ma trận Q nhanh chóng mất đi tính trực giao hoàn hảo.
- **Tốc độ hội tụ của thuật toán Lặp QR:** Do giới hạn yêu cầu tự cài đặt, thuật toán Lặp QR thuần túy (không có cơ chế dịch chuyển Shifts) mất rất nhiều thời gian (hàng trăm vòng lặp) mới có thể hội tụ khi gặp các trị riêng nằm quá sát nhau.
- **Trực quan hóa không gian 3D:** Thư viện Manim có learning-curve (đường cong học tập) khá dốc. Việc đồng bộ hóa các phép chiếu toán học với hoạt ảnh (animations) của camera 3D tốn rất nhiều thời gian canh chỉnh tọa độ.

- **Kiến thức khó:** Đồ án có nhiều phần kiến thức khó, đòi hỏi phải nghiên cứu nhiều tài liệu tham khảo và tham khảo nhiều nguồn trên Internet, khó khăn nhất là cài các thuật toán mà không sử dụng thư viện Numpy.

3. Bài học rút ra

Sau quá trình cùng nhau hoàn thành đồ án thì nhóm có rút ra được một số bài học cho mình:

1. **Khoảng cách giữa lý thuyết và thực hành:** Một công thức toán học đúng hoàn toàn trên giấy (như Gram-Schmidt) chưa chắc đã là một thuật toán tốt khi đưa vào bộ nhớ dấu phẩy động (Float 64-bit) của máy tính. Việc hiểu rõ giới hạn phần cứng giúp nhóm có tư duy lập trình phòng thủ tốt hơn.
2. **Hiểu sâu hơn về các thư viện công nghiệp:** Qua việc tự cài đặt và liên tục nhận kết quả thua kém so với NumPy ở các test cases khó, nhóm đã hiểu được tại sao các thư viện hiện đại phải sử dụng những thuật toán phức tạp hơn rất nhiều như Householder Reflections (để phân rã) và Wilkinson Shifts (để chéo hóa).
3. **Kỹ năng:** Qua quá trình cùng nhau làm và nghiên cứu các tài liệu, từng thành viên trong nhóm đồng thời phát triển nhiều kỹ năng mềm như: research, làm việc nhóm, giao tiếp. Các kỹ năng trên đều là kỹ năng nền tảng quan trọng cho tương lai sau này.

TÀI LIỆU THAM KHẢO

- [1] G. Strang, *Introduction to Linear Algebra*, 6th. Wellesley-Cambridge Press, 2023.
- [2] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*. SIAM, 1997.
- [3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th. Johns Hopkins University Press, 2013.
- [4] NumPy Developers. “Numpy reference documentation - linear algebra (numpy.linalg),” Accessed: May 10, 2024. [Online]. Available: <https://numpy.org/doc/stable/reference/routines.linalg.html>.
- [5] Manim Community Developers. “Manim community documentation - mathematical animation framework,” Accessed: May 10, 2024. [Online]. Available: <https://docs.manim.community/>.

PHỤ LỤC

Danh sách Link Video Demo

Dưới đây là các đường dẫn video trực quan hóa thuật toán bằng thư viện Manim và các kết quả demo của nhóm:

- **Video 1 (Minh họa Phép khử Gauss):** <https://youtube.com/...>
- **Video 2 (Trực quan hóa Phân rã QR):** <https://youtube.com/...>
- **Video 3 (Chéo hóa ma trận):** <https://youtube.com/...>